

Machine Learning for MRI Reconstruction

11

Kerstin Hammernik^{a,d}, Thomas Küstner^{b,d}, and Daniel Rueckert^{a,c}

^aTechnical University Munich, AI in Healthcare and Medicine, Klinikum Rechts der Isar, Munich, Germany

^bUniversity Hospital Tübingen, Medical Image and Data Analysis, Department of Radiology, Tübingen, Germany

^cImperial College London, Department of Computing, London, United Kingdom

11.1 Introduction

The development of Artificial Intelligence (AI) and Machine Learning (ML) has evolved tremendously during the last decade. In the field of medicine, including in the field of medical imaging, AI and ML have emerged as techniques that are likely to transform clinical practice fundamentally in the coming years. In particular in medical imaging, AI and ML have the potential to impact all stages of the imaging pipeline, from (1) image acquisition and reconstruction to (2) image analysis and interpretation and (3) diagnosis and prognosis. The terms AI, ML, and more recently Deep Learning (DL) are used as synonymous, however, there is a substantial difference between these terms.

Artificial intelligence

In 1956, the term *Artificial Intelligence (AI)* was formulated by John McCarthy as “the science and engineering of making intelligent machines” at the 1956 Dartmouth Conference [1]. Since then, AI has served as an umbrella term to solve any task that usually requires human intelligence. Inspired by the biological neuron, Frank Rosenblatt introduced the perceptron [2], a single artificial neuron.

Machine learning

ML is a subbranch of AI that describes any *data-driven* learning model to predict the output of a particular task and summarizes (statistical) methods that enable machines to improve performance with experience. After the AI winter, ML started to flourish in the 1980s with developments in Boltzmann machines [3], recurrent neural networks [4–7], multilayer perceptron, and back-propagation [8]. The neocognitron [9] reported in 1980 can be seen as the origin of Convolutional Neural Networks (CNNs) [10]. All these publications have formed the basis for later developments in the field of DL.

The basic working principle of ML is summarized in Fig. 11.1. The ML algorithm receives as input *features* or a reduced/combined subset of features that are extracted from input data. These input data include acquired sensor signals (e.g., raw MR *k*-space), (reconstructed) images, clinical and anthropometric meta-parameters, among many others. Feature extraction requires hand-crafting and manual

^d Equal contribution.

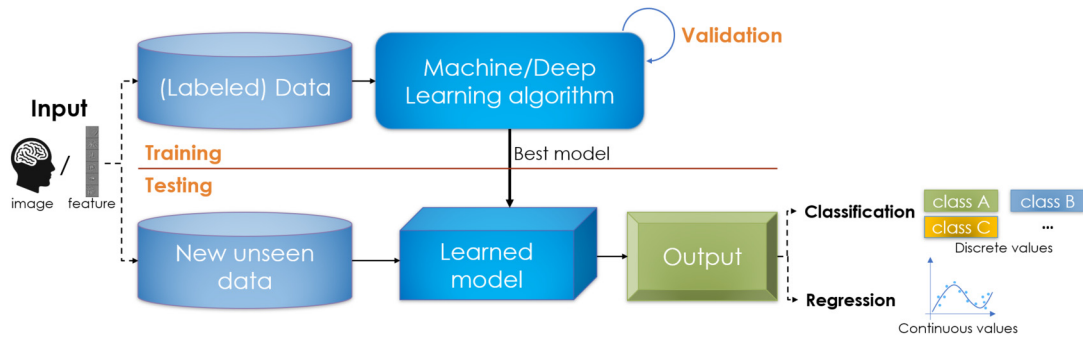


FIGURE 11.1 Processing Steps in Machine Learning (ML)

Processing steps of ML algorithms with corresponding images or feature inputs and task-specific outputs of classification or regression. Algorithms are learned on labeled (supervised) or unlabeled (unsupervised) data in a training step. Validation with independent datasets make possible tracking and optimization of learning. Testing is performed on new unseen data to produce the best trained model.

finetuning of features according to the underlying task and application. Hence, the quality of the predicted output depends not only on the ML algorithm itself but also on the extracted features.

Deep learning

DL [11] is a subbranch of ML that enables automated feature extraction and comprise deep NNs consisting of multiple layers. The basics of these methods were established in the late 1980s but only gained widespread attention with the remarkable performance boost of AlexNet [12] in the ImageNet Challenge 2012 for object recognition. The increasing availability and performance of graphics processing units (GPUs), as well as annotated large-scale databases, helped in the development of novel neural-network building blocks (see Section 11.5) and deep architectures (see Section 11.6).

11.2 Organization of this chapter

In this chapter we focus on the application of ML in the context of Magnetic Resonance Imaging (MRI) reconstruction. ML enables to overcome common challenges in regularized undersampled reconstruction approaches, such as Compressed Sensing (CS) approaches (Chapter 8) that require long reconstruction times and the selection of a hand-crafted sparsifying transform by using data-driven features. To introduce the important elements of building a NN for MR reconstruction, we introduce a general notation to deal with both the complex-valued nature of MR signals, real-valued magnitude images, and the dimensionality of input data in Appendix 11.A. In Section 11.3, we introduce various types of learning and provide insights into training, validating, and testing NNs. To study the application potential of ML in MRI reconstruction, we introduce various tasks in Section 11.4. Basic layers of a NN are presented in Section 11.5 that can be bundled to larger networks outlined in Section 11.6. Finally,

we use the introduced concepts to build own reconstruction networks in the Tensorflow¹/Keras² framework. Practical information regarding the implementation in the various frameworks are highlighted in gray boxes throughout this chapter.

11.3 Machine learning definitions

In the following, we introduce important ML definitions. Here, we build on the notation for both real-valued and complex-valued data, introduced in Appendix 11.A.

11.3.1 Learning models

The learning model represents any (non)linear and parametric model

$$\mathbf{x}_{\text{NN}} = f_{\boldsymbol{\theta}}(\mathbf{x}) \quad (11.1)$$

that maps the inputs $\mathbf{x}$ to the neural network outputs $\mathbf{x}_{\text{NN}}$. The mapping function $f_{\boldsymbol{\theta}}$ represents a NN whose parameters $\boldsymbol{\theta}$ can be optimized under some given cost function $\mathcal{L}$ (see Section 11.3.3). During a training process, the model sees different data samples such that the parameters $\boldsymbol{\theta}$ of the NN can be learned to predict a reasonable output $\mathbf{x}_{\text{NN}}$ for any new input $\mathbf{x}$. During testing, the model is fixed and the trained parameters generate the output from new unseen test data. The models perform a sequence of operations on the inputs to yield a task-specific output. The most commonly used learning models and their basic building blocks are summarized in Section 11.5. The reader is referred to textbooks [13–15] for more details on learning models.

11.3.2 Types of learning

Types of learning can be differentiated based on the availability of label information and the type of label integration during training. In principle this relates to how closely a human observer is involved during training.

In *supervised learning*, data samples along with their task-specific labels exist in the database. However, labeling is very time- and cost-intensive and often requires human interaction, ranging from data sorting to curating to annotating structures within the image. For example, in undersampled MR image reconstruction, a labeled ground-truth corresponds to a fully sampled dataset or a dataset acquired with mild undersampling.

In *semi-supervised learning*, both labeled and unlabeled data are included in the learning process. The unlabeled data provides additional information about the underlying data distribution.

Self-supervised learning circumvents the problem of external labels. The input data itself is used to guide the learning. In a similar sense, in *reinforcement learning* the model receives feedback, i.e., rewards or penalties, based on its current prediction, that drives the training procedure.

¹ <https://www.tensorflow.org/>.

² <https://keras.io/>.

Active learning integrates an oracle into the training procedure, which is periodically queried to either label or select the next most meaningful samples for training. The oracle is in most cases a human observer, but can also be another algorithm [16].

In *unsupervised learning*, no labeled data or any type of a-priori information about the data is available that could be leveraged to guide the training. The network learns to identify patterns in the data. Common approaches are clustering [13] (e.g., k-means or Gaussian mixture models), Principal Component Analysis (PCA), (variational) autoencoders [17,18], Deep belief networks [19] or Generative Adversarial Networks (GANs) [20].

Transfer learning investigates the possibility to transfer knowledge between models or tasks. It can involve sharing information from simpler to more complex tasks, or from a source domain to another (but similar) target domain [21–23].

Federated learning [24–26] trains a model across multiple decentralized devices, where each device holds its own set of training data and only the model weights are shared across devices. This allows for training across highly heterogeneous datasets. Furthermore, federated learning has a vast potential in medicine, where sharing of data across multiple centers is challenging due to data protection and data privacy [27].

11.3.3 Cost function, optimization and backpropagation

Cost function

The *cost function* (*error*, *loss*) $\mathcal{L}$ is a quantitative measure that describes how well the neural network $f_{\theta}(\mathbf{x})$ predicts the output $\mathbf{x}_{\text{NN}}$ from the input $\mathbf{x}$. In supervised learning, the aim is to minimize the loss between prediction and label/ground-truth. In general, the ML optimization task can be formulated as

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \sum_j \lambda_j \mathcal{L}_j(\theta), \quad (11.2)$$

which is composed of several λ_j weighted sub-losses $\mathcal{L}_j(\theta)$. This can include, e.g., regularization of weights or penalties on intermediate model outputs. Note that the result of a loss function is a real-valued scalar, even if the input data $\mathbf{x}$ and model output $\mathbf{x}_{\text{NN}}$ are in the complex domain as in MRI reconstruction.

Depending on the chosen architecture (see Section 11.6) and defined task (see Section 11.4), the appropriate losses are selected. Common cost functions in linear-regression problems are the mean-squared-error (MSE, L_2 distance)

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{2N_s} \sum_{i=1}^{N_s} \|f_{\theta}(\mathbf{x}_i) - \mathbf{x}_{\text{GT},i}\|_2^2 \quad (11.3)$$

or the mean-absolute error (MAE, L_1 distance)

$$\mathcal{L}_{\text{MAE}}(\theta) = \frac{1}{N_s} \sum_{i=1}^{N_s} \|f_{\theta}(\mathbf{x}_i) - \mathbf{x}_{\text{GT},i}\|_1, \quad (11.4)$$

to minimize the error between predicted output $f_{\theta}(\mathbf{x})$ and ground-truth $\mathbf{x}_{\text{GT}}$ for all N_s data samples. Differences between commonly used loss functions, including MSE, MAE, or Structural Similarity

Index (SSIM) [28], can be found in [29]. Practically, any piece-wise differentiable function can be used as a loss function.

Perceptual or style-transfer losses are often derived from feature maps of pre-trained networks and try to mimic the human visual system by capturing differences between high-level image feature representations [30]. Adversarial losses (minimax loss [20] or Wasserstein loss [31]) of GANs try to replicate the probability distribution of the input data and incorporate this information into training. Losses can also directly operate on the latent space, i.e., space in which feature maps lie, such as Kullback–Leibler divergence [18] to model feature distributions in deeper layers.

Box 11.1 Implementation of losses in ML frameworks

Common loss functions are provided in `tf.keras.losses`. A loss function needs to be defined in order to compile a model. Additional losses on, e.g., intermediate outputs of the model, can be defined in the model using the `add_loss()` method in the `call` method of a layer or sub-classed model. Weight regularization applies penalty on the network parameters. Weight regularization is defined in `tf.keras.regularizers` and can be added when creating a `tf.keras.layer`. The Keras API collects the defined losses in the model.

Optimization

Once a cost function is defined, the model parameters θ need to be found that minimize the cost function $\mathcal{L}$. Gradient descent algorithms are commonly used to perform this optimization. While the negative gradient of the loss function $-\frac{\partial \mathcal{L}}{\partial \theta}$ points towards the local minimum, optimal parameters θ can be found by iteratively going along this direction with a pre-defined step size τ , known as learning rate. A simple gradient descent algorithm iteratively solves for

$$\theta^{[it+1]} = \theta^{[it]} - \tau \frac{\partial \mathcal{L}}{\partial \theta}. \quad (11.5)$$

Key challenges for gradient-based optimizations are the proper selection of the learning rate τ to reach the (global) minimum and to avoid getting trapped in numerous suboptimal local minima for the commonly nonconvex optimization problem. Many other *optimizers* exist to toggle these challenges in various ways. They differ in the way how much data is incorporated to update the parameters if gradient momentum is included and/or if the learning rate is adapted, known as learning-rate scheduling [32].

The most common optimizers in ML are stochastic gradient descent (SGD) [33], Adaptive moment estimation (ADAM) [34], RMSProp [35], Nesterov accelerated gradient (NAG) [36], Adagrad [37], Adadelta [38], AdaMax [34], or Nesterov-accelerated Adaptive Moment Estimation (NADAM) [39]. An overview and comparison of these optimizers can be found in [40].

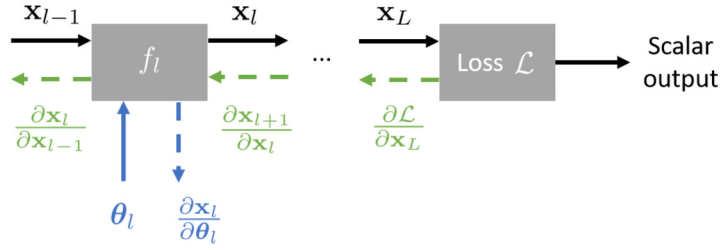
Box 11.2 Implementation of optimizers in ML frameworks

Common optimizers are provided in `tf.keras.optimizers`. An optimizer needs to be defined in order to compile a model. A learning-rate scheduler (see `tf.keras.optimizers.schedules`) can be added to the optimizer call.

Back-propagation

In general, the network f_{θ} consists of L connected layers

$$\mathbf{x}_{\text{NN}} = f_{\theta}(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}) = \mathbf{x}_L. \quad (11.6)$$

**FIGURE 11.2 Back-propagation**

The network consists of L connected layers with $f_l = \mathbf{x}_l$ being the output of the l -th layer, $1 \leq l \leq L$. The output prediction $\mathbf{x}_{\text{NN}} = \mathbf{x}_L$ is passed to a loss function $\mathcal{L}$. This gives an error that is back propagated using the chain rule of differentiation to update the network parameters θ_l .

The output of the l -th layer is denoted as $\mathbf{x}_l = f_l(\cdot)$, $1 \leq l \leq L$. In training, the parameters θ of the network f_θ are learned to minimize the loss function Eq. (11.2). This involves two steps as depicted in Fig. 11.2. First a feed-forward prediction from the input $\mathbf{x}$ to the output $\mathbf{x}_{\text{NN}} = f_\theta(\mathbf{x})$ is required to evaluate the loss $\mathcal{L}$ for the current set of parameters θ . Second, the error between the predicted output $f_\theta(\mathbf{x})$ and the ground-truth labels $\mathbf{x}_{\text{GT}}$ is back propagated to adapt the parameters θ . Back propagation [41] can be realized by applying the chain-rule of differentiation

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{NN}}} \frac{\partial \mathbf{x}_{\text{NN}}}{\partial \theta}. \quad (11.7)$$

More specifically, the gradient needs to be back propagated from the output through the single layers to update the layer parameters θ_l . Together with Eq. (11.5), the update rule for the parameters θ_l of layer f_l in the $[it]$ -th training iteration becomes

$$\begin{aligned} \theta_l^{[it+1]} &= \theta_l^{[it]} - \tau \frac{\partial \mathcal{L}}{\partial \theta_l} \\ &= \theta_l^{[it]} - \tau \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}} \cdots \frac{\partial \mathbf{x}_l}{\partial \theta_l}. \end{aligned} \quad (11.8)$$

If complex-valued layers and network outputs $\mathbf{x}_{\text{NN}}$ are involved, complex back propagation following Wirtinger calculus (see Appendix 11.B) has to be considered.

Box 11.3 Back-propagation in practice

Automatic differentiation is supported in ML frameworks. Complex back propagation according to Wirtinger calculus is supported in Tensorflow/Keras. The most recent versions of PyTorch now also support complex-valued processing.

11.3.4 Training, validation, and testing

Training describes the process of updating the model parameters θ in the ML model $f_\theta(\mathbf{x})$ optimizing a cost function. The training database is the composition of N_{train} training samples. In case of super-

vised learning, the cost function involves the labels / reference output $\mathbf{x}_{\text{GT}}$, hence, the training database contains both training samples and their associated annotated labels. The gradient descent in Eq. (11.8) would require all training samples for the update, however, this requires a huge computing effort especially for large datasets. In ML, stochastic optimization is used to overcome this computational burden. SGD (the stochastic version of GD) computes a single gradient update only on a subset of training samples, provided in a single batch. The *batch size* N_b is set a priori. For N_{train} training samples and a batch size N_b , we obtain $\lfloor N_{\text{train}}/N_b \rfloor$ iterations per epoch. An epoch refers to one pass over the full training set. The amount of epochs determines the training duration. Careful consideration needs to be taken in terms of the amount of training data, model capacity, and problem complexity to avoid under- or overfitting which can be observed by comparing discrepancies between the *training* and *validation loss* [15]. During training, dropout layers (see Section 11.5.7) can be included in NNs, weights can be regularized or early stopping can be deployed to reduce the risk of overfitting. The model behavior can be observed by the training loss. However, it is advisable to monitor the training progress with a separate validation set.

Validation is performed during model training to track the model performance or to optimize model hyperparameters, but not the model parameters θ themselves. Therefore, a separate validation set N_{val} data samples with annotated labels (in case of supervised learning) is used to evaluate the performance of the currently learned model following a predetermined validation cycle. Evaluation metrics are, on one hand, the loss function itself (validation loss), but can also include other quantitative measures (MSE, MAE, SSIM) [29], on the other.

In *testing/inference/prediction*, the trained model is tested with N_{test} newly unseen data samples. Test data is usually not accompanied by annotated labels (in case of supervised learning). In case annotated labels or ground truth references are available, nonblinded similarity measures (e.g., MSE) can be applied to assess the model performance.

11.3.5 Database splitting

Training, validation, and test sets are disjoint. Available data samples N_s can be split up into training N_{train} , validation N_{val} , and test set N_{test} according to different inclusion/exclusion criteria or completely random. Database creation and curation is an important step towards a reliable ML method. The outline of the database specifies the structure of the database pipeline needed to load the data into the ML application (see Section 11.7.3).

Selection of representative training samples N_{train} is crucial to provide enough diversity for training, but with sufficient consistency to converge. The validation set size N_{val} is usually much smaller than N_{train} , e.g., 10%–20% of the number of training samples, depending on the underlying application, task, and hyper-parameter optimization. Test samples N_{test} are never seen by the method during training and ideally (but not necessarily) reflect the same underlying trained application that enables inference of performance and robustness and/or generalization of the model.

Random splitting

While randomly splitting data samples into training, validation, and test set (or between training and validation set) is feasible and justified for some applications, it is strongly discouraged in the field of medical imaging. A random splitting ignores the information from which patient the data sample is derived and hence samples from a specific patient can be assigned to training, validation, and test set that in the end potentially boosts performance, but definitively biases the results.

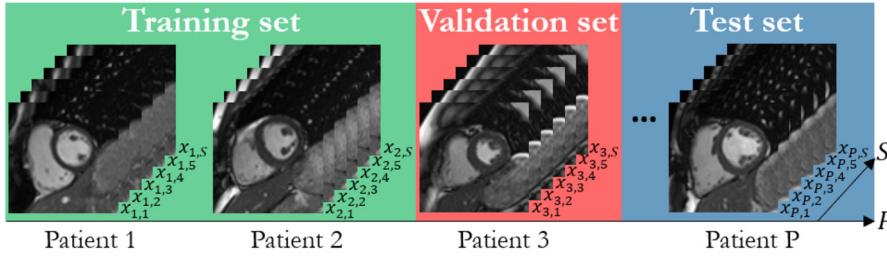


FIGURE 11.3 Database splitting with patient leave-out

Splitting of available data in training, validation, and test set. Each of the P patients has S samples (slices in a volume, imaging contrasts, ...). A patient leave-out approach is chosen, i.e., unique patients are assigned for each set.

Patient leave-out approach

Therefore, common splitting methods consider a patient leave-out approach, i.e., only unique patients are fed through the model in training, validation, and testing. This avoids any bias of the model towards specific anatomies or pathologies. A database of P patients with each patient having S data samples (e.g., slices in a volume), consists of $N_s = P \cdot S$ samples

$$\left\{ \underbrace{x_{1,1}, \dots, x_{1,S}}_S, \underbrace{x_{2,1}, \dots, x_{2,S}}_S, \underbrace{x_{3,1}, \dots, x_{3,S}}_S, \dots, \underbrace{x_{P,1}, \dots, x_{P,S}}_S \right\}. \quad (11.9)$$

In a patient leave-out approach, unique patients $\{x_{i,1}, \dots, x_{i,S}\}$ are assigned for the training, validation, and test set as illustrated in Fig. 11.3. In a similar fashion, one could also define other and/or additional means to sort and split the data along S , e.g., imaging contrast, multi-parametric information, trajectory, etc., depending on the specific reconstruction problem.

Cross-validation

For smaller databases, it is advisable to perform a cross-validation in order to reduce bias towards selected samples and investigate the method's robustness. In a k -fold cross-validation, the available database of N_s samples is split into k folds—considering unique patients are sorted into each fold—with each having N_s/k samples. Assignment of samples to these folds can either be random (after patient consideration) or follow specific criteria. In total, k networks are then trained, one for each of the folds. Deviation in evaluation metrics between folds gives an indicator on the method's robustness.

11.4 Task definition for MR reconstruction

ML has the ability to learn expressive feature extractors from data and enables us to solve tasks that are challenging to solve with fixed, hand-crafted models. Various types of tasks can be solved using ML such as image segmentation, image classification, and regression tasks. For *image classification*, the task of the NN is to learn a function $f_\theta(\mathbf{x}) : \mathbb{K}^N \rightarrow \{1, \dots, L\} \triangleq [L]$ to assign a label l to the input $\mathbf{x}$,

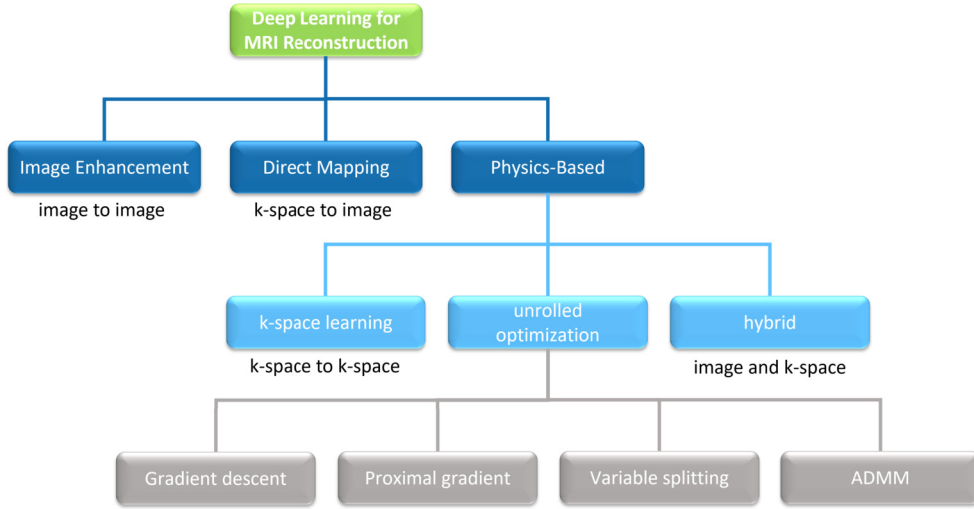


FIGURE 11.4 MR reconstruction task definition

Classification of ML reconstruction tasks that differ in terms of input and targeted application output.

where $\mathbb{K}$ is an arbitrary field. An example is tumor classification with the labels *benign* and *malign*. For *image segmentation* tasks, the network $f_{\theta}(\mathbf{x}) : \mathbb{K}^N \rightarrow [L]^N$ assigns a label l to every pixel of the input vector $\mathbf{x}$, hence, it partitions the image $\mathbf{x}$ into a set of L distinct classes/objects, e.g., organs.

Image classification and segmentation assign a global or local label to the input vector $\mathbf{x}$. In contrast, the function $f_{\theta}(\mathbf{x}) : \mathbb{K}^N \rightarrow \mathbb{K}^{N_d}$ predicts a numerical value for a given input vector $\mathbf{x}$ in an (*image*) *regression* task, where N_d is the output dimension. Image-to-image translation describes an image regression task if continuous predictions are assigned to every pixel in the image. MRI reconstruction can be viewed as sensor-to-image translation task.

As previously discussed in Chapter 2, in MR image reconstruction, we aim at recovering an image $\hat{\mathbf{x}}$ from the k -space signal $\mathbf{s}$ that is corrupted by measurement noise ϵ following

$$\mathbf{s} = \mathbf{E}\mathbf{x} + \epsilon, \quad (11.10)$$

where $\mathbf{E} \in \mathbb{C}^{K \times N}$ is the linear forward operator describing the MR acquisition model and K denotes the dimensionality of the underlying k -space data. Depending on the imaging application and signal modeling, the operator $\mathbf{E}$ involves Fourier transforms, sampling trajectories, and coil sensitivity maps. Field inhomogeneities, relaxation effects, motion, and diffusion can be also considered (Chapters 12–16), which may lead to a nonlinear measurement operator.

In ML frameworks, mapping functions f_{θ} can be used in various ways to reconstruct an image $\mathbf{x}$ from the measured data. All tasks have an image $\mathbf{x}$ and/or k -space $\mathbf{s}$ as input to the NN function f_{θ} , but can also include further MR specific information as meta parameters or other tensors, e.g., trajectories and coil sensitivity maps.

An overview of common ML reconstruction tasks for MRI, differing in terms of input and targeted application output is shown in Fig. 11.4. These reconstruction tasks are further described hereafter.

A summary of recently proposed ML-based MR reconstruction approaches is depicted in Table 11.1, along with details about the underlying task, data processing, input dimensionality, and handling of a complex domain. MRI data can be processed in a coil-combined/single-coil setting or as multicoil data. While multicoil data are usually acquired in practice and provide a real-time scenario, single-coil/coil-combined data reduce the computational burden substantially.

11.4.1 Image enhancement

Certain types of undersampling, such as pseudorandom, radial, or spiral sampling, introduce incoherent noise-like aliasing in the zero-filled reconstructed images, while we observe coherent undersampling artifacts from regular Cartesian sampling. Thus, an *image enhancement* task can be used to reduce the aliasing in the images. The function $f_{\theta}(\mathbf{x}) : \mathbb{K}^N \rightarrow \mathbb{K}^N$ performs image-to-image regression by predicting the output value $\mathbf{y}$ based on the corrupted input image $\mathbf{x}$. The input to the enhancement task can be the zero-filled (and noise-affected) MR images or reconstructed MR images that present remaining aliasing or noise amplification for large undersampling factors, e.g., images reconstructed with parallel imaging. Instead of learning the enhanced image, some approaches learn the residual noise to be removed from the noisy input [42,43]. The mapping f_{θ} only acts on the image $\mathbf{x}$ and has no information of the acquired rawdata. Hence, consistency to the measured k -space signal $\mathbf{s}$ cannot be guaranteed. Approaches exist that add additional k -space consistency to the cost function [44] or enforce k -space consistency after image enhancement [45].

11.4.2 Direct k -space to image mapping

Another ML-based approach is to reconstruct the MR image directly from the acquired k -space data. With this approach, the so-called direct k -space to image mapping, the k -space data are fed directly into the mapping function to achieve $\mathbf{y} = f_{\theta}(\mathbf{s})$. Consequently, the mapping function f_{θ} approximates the forward model. Learning a direct mapping is especially useful if the forward model or parts of the forward model are not exactly known. In the case of fully sampled MRI under ideal conditions, the learned mapping approximates the Fourier transform [46]. However, this becomes computationally very demanding due to fully connected layers (see Section 11.5.4) that are involved here. Furthermore, consistency to the acquired k -space data cannot be guaranteed.

11.4.3 Physics-based reconstruction

Another family of ML-based MR reconstruction methods is referred as physics-based reconstruction. These approaches integrate the traditional physics-based modeling of the MR encoding with ML, ensuring consistency to the acquired data. We can distinguish two classes of problems: (1) learning in k -space domain and (2) iterative optimization in image domain with interleaved data consistency steps. The first approaches are referred to as k -space learning, whereas the latter one are known as unrolled optimization methods. These two approaches can be combined to *hybrid* approaches that learn both a NN in k -space domain and image domain.

Unrolled optimization

Physics-based learning, which is modeled as iterative optimization, can be viewed as generalization of iterative SENSE [47,48] (Chapter 6) with a learned regularization in image domain. To explore this in

more detail, let us recap the basic variational image reconstruction problem defined in (Chapter 2)

$$\mathbf{x}^* \in \arg \min_{\mathbf{x} \in \mathbb{K}^n} \frac{\lambda}{2} \|\mathbf{E}\mathbf{x} - \mathbf{s}\|_2^2 + R[\mathbf{x}], \quad (11.11)$$

which contains a data-consistency term $D[\mathbf{E}\mathbf{x}, \mathbf{s}] = \frac{\lambda}{2} \|\mathbf{E}\mathbf{x} - \mathbf{s}\|_2^2$ and a regularization term $R[\mathbf{x}]$, that imposes prior knowledge on the reconstruction $\mathbf{x}$. As discussed in Chapters 2 and 3, the easiest way to solve Eq. (11.11) is to use a gradient descent scheme to optimize for the reconstruction $\mathbf{x}$. This results in

$$\mathbf{x}^{[it+\frac{1}{2}]} = \mathbf{x}^{[it]} - \tau^{[it]} \mathbf{E}^H (\mathbf{E}\mathbf{x}^{[it]} - \mathbf{s}) \quad (11.12)$$

$$\mathbf{x}^{[it+1]} = \mathbf{x}^{[it+\frac{1}{2}]} - \tau^{[it]} \nabla_{\mathbf{x}} R[\mathbf{x}] \quad (11.13)$$

where τ is the (iteration-dependent) step size. Assuming $R[\mathbf{x}]$ is a fixed, convex function, the gradient descent scheme would be run for a large number of iterations. In learning algorithms, we unroll this scheme for a fixed number of T iterations to obtain a solution for $\mathbf{x}$. An NN replaces the gradient of the handcrafted regularizer by a data-driven mapping function f_{θ} . Thus, training several iterations with alternating mapping functions f_{θ} and intermittent data consistencies reflect *unrolled optimizations* [49]. For a GD scheme, this results in

$$\mathbf{x}^{[it+\frac{1}{2}]} = \mathbf{x}^{[it]} - \tau^{[it]} \mathbf{E}^H (\mathbf{E}\mathbf{x}^{[it]} - \mathbf{s}) \quad (11.14)$$

$$\mathbf{x}^{[it+1]} = \mathbf{x}^{[it+\frac{1}{2}]} - \tau^{[it]} f_{\theta}(\mathbf{x}). \quad (11.15)$$

Another example would be to learn a proximal gradient scheme (Chapter 3). A common way in MRI reconstruction is to first take a step in the direction of the regularization term and then perform the proximal mapping with respect to the data term [50].

$$\mathbf{x}^{[it+\frac{1}{2}]} = \mathbf{x}^{[it]} - \tau^{[it]} \nabla_{\mathbf{x}} R[\mathbf{x}^{[it]}], \quad (11.16)$$

$$\mathbf{x}^{[it+1]} = \text{prox}_{\tau^{[it]} \frac{\lambda}{2} (\mathbf{E}, \mathbf{s})} \left(\mathbf{x}^{[it+\frac{1}{2}]} \right) = \arg \min_{\mathbf{x} \in \mathbb{K}^n} \frac{1}{2\tau^{[it]}} \left\| \mathbf{x} - \mathbf{x}^{[it+\frac{1}{2}]} \right\|_2^2 + \frac{\lambda}{2} \|\mathbf{E}\mathbf{x} - \mathbf{s}\|_2^2. \quad (11.17)$$

Here, again the gradient, with respect to the regularization term, can be replaced by a mapping function f_{θ} yielding

$$\mathbf{x}^{[it+\frac{1}{2}]} = \mathbf{x}^{[it]} - \tau^{[it]} f_{\theta}(\mathbf{x}^{[it]}), \quad (11.18)$$

$$\mathbf{x}^{[it+1]} = \text{prox}_{\tau^{[it]} \frac{\lambda}{2} (\mathbf{E}, \mathbf{s})} \left(\mathbf{x}^{[it+\frac{1}{2}]} \right) = \arg \min_{\mathbf{x} \in \mathbb{K}^n} \frac{1}{2\tau^{[it]}} \left\| \mathbf{x} - \mathbf{x}^{[it+\frac{1}{2}]} \right\|_2^2 + \frac{\lambda}{2} \|\mathbf{E}\mathbf{x} - \mathbf{s}\|_2^2. \quad (11.19)$$

While for single-coil imaging a closed-form solution for the proximal mapping exists [51], the proximal mapping for multicoil imaging can be solved using a conjugate gradient optimizer [50]. In practice, any other optimization can be used for algorithm unrolling, such as variable splitting [52] or primal-dual optimization [53]. While this section gave a very general mathematical overview, the implementation of MRI-specific data-consistency layers are described in Section 11.5.11.

Plug-and-play priors

Trained image denoisers can be also combined with physics-based learning (see Section 11.4.3) or conventional reconstructions (see Chapter 2) and serve thus as an advanced regularization for traditional optimization problems. Specifically, it was noticed by Heide et al. [54] and Venkatakrishnan et al. [55] that advanced denoising schemes such as BM3D [56] and Nonlocal means [57] can be used efficiently with multiple forward models. Inspired by [54,55], the fixed denoiser has been replaced by data-driven CNNs in computer vision [58] and MRI [59,60]. The advantage of plug-and-play priors is that the denoiser can be trained off-line and does not have to involve any forward model. The trained denoiser can then be matched with any optimization scheme, e.g., ADMM [55] or Proximal gradient optimization [58].

k -space learning

A prominent approach for physics-based learning in k -space domain [61] can be viewed as an extension of the linear kernel estimation in GRAPPA (Chapter 6). Here, a nonlinear kernel modeled by the mapping function f_θ is learned from the ACS. The missing k -space lines can then be filled using this estimated, nonlinear kernel and the data is then transformed to the image space using an inverse Fourier transform. The final image is obtained by root-sum-of-squares reconstruction of the individual coil images.

Hybrid learning

The last type of approaches we present here are *hybrid* approaches [62] that combine the advantages of learning in k -space domain and learning in image domain. These networks are applied in an alternating manner to obtain the final reconstruction $\mathbf{x}$. When designing hybrid approaches, it is important to keep the basic theorems of the Fourier transform in mind: local changes in image domain result in global changes in k -space domain and vice versa, to avoid unexpected behavior.

11.5 Core concepts: layers

NNs are composed of several layers. Each layer operates on received input variables and output variables, which are denoted by $\hat{\mathbf{x}}$ in the following. A layer may have trainable and non-trainable parameters. Various types of layers are commonly used in ML-based MR reconstruction, including convolution, normalization, activation, among others. These layers are further described hereafter.

11.5.1 Convolution layer

Convolutions are linear operations that extract *features* of an image. While shallow convolution layers extract low-level features such as edges, deeper layers extract more abstract, high-level features such as shapes or objects. A convolution layer maps $N_{f,\text{in}}$ input feature channels to $N_{f,\text{out}}$ output feature channels

The discrete convolution of an image $\mathbf{x}$ with filter kernels $\mathbf{k}_{i,j}$ reads as

$$\hat{\mathbf{x}}_j = \sum_{i=1}^{N_{f,\text{in}}} \mathbf{x}_i * \mathbf{k}_{i,j} = \mathbf{K}_{i,j} \mathbf{x}_i, \quad j = 1, \dots, N_{f,\text{out}}, \quad (11.20)$$

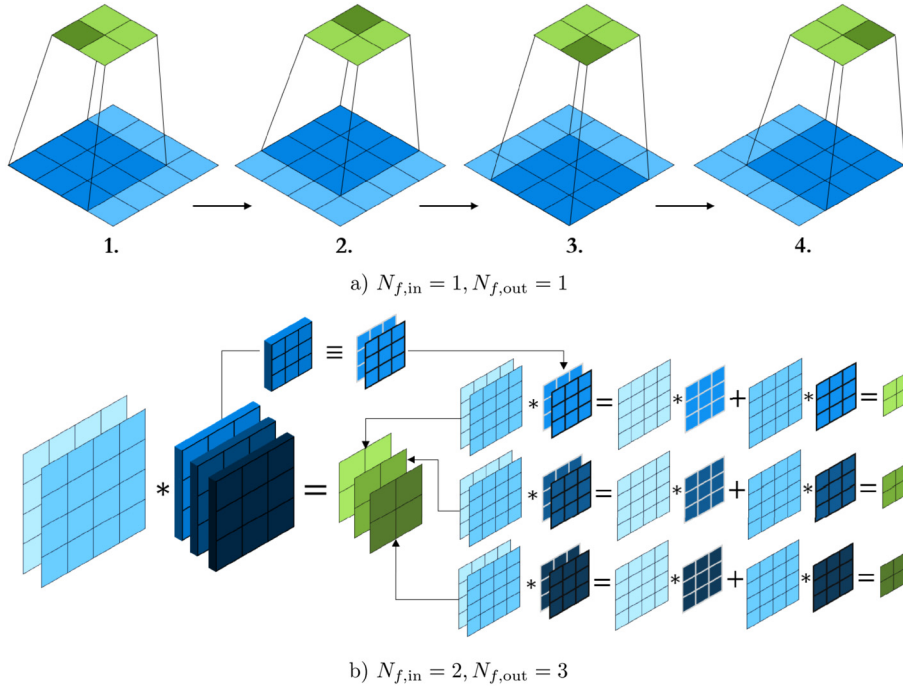


FIGURE 11.5 Convolution operation

The input $\mathbf{x}$ (light blue (light gray in print version)) is convolved with the filter kernel $\mathbf{k}$ (dark blue (dark gray in print version)) producing the output feature map (green (gray in print version)). The filter kernel of size 3×3 is slid over the entire input of size 5×5 , yielding a scalar estimate (dark green (mid gray in print version)) at each of the four positions of the 2×2 output feature map. No padding ($p = 0$), a stride of $s = 1$ with dilation rate $d = 1$, was applied in all directions. Convolution operation for learning **a**) a single filter, i.e., output channel $N_{f,\text{out}} = 1$ from a single channel input $N_{f,\text{in}} = 1$ and b) three filters, i.e., output channel $N_{f,\text{out}} = 3$ from a two channel input $N_{f,\text{in}} = 2$. In this case for each input and output channel combination, a kernel is learned, i.e., $2 \cdot 3 = 6$ kernels of size 3×3 are trained. Input channels are convolved with the respective kernel and summed up to form the 2×2 output for each of the three output channels.

where the subscripts denote the feature channels. This produces $N_{f,\text{out}}$ feature maps in $\mathbf{x}_{\text{NN}}$. The convolution with a filter kernel $\mathbf{k}_{i,j}$ can be represented as a matrix–vector multiplication, where $\mathbf{K}_{i,j} : \mathbb{K}^N \rightarrow \mathbb{K}^N$ is a sparse matrix. Note that convolution filters are local feature extractors and invariant to translation. The convolution operation is depicted in Fig. 11.5.

For real-valued convolutions, it is $\mathbb{K} = \mathbb{R}$. For complex convolutions, $\mathbb{K} = \mathbb{C}$, the convolution operation is extended to

$$\begin{aligned} \mathbf{x}_i * \mathbf{k}_{i,j} &= (\text{Re}(\mathbf{x}_i) * \text{Re}(\mathbf{k}_{i,j}) - \text{Im}(\mathbf{x}_i) * \text{Im}(\mathbf{k}_{i,j})) \\ &\quad + i \cdot (\text{Im}(\mathbf{x}_i) * \text{Re}(\mathbf{k}_{i,j}) + \text{Re}(\mathbf{x}_i) * \text{Im}(\mathbf{k}_{i,j})) \\ &= \mathbf{K}_{i,j} \mathbf{x}_i \end{aligned}$$

$$\begin{aligned}
&= (Re(\mathbf{K}_{i,j}) \cdot Re(\mathbf{x}_i) - Im(\mathbf{K}_{i,j}) \cdot Im(\mathbf{x}_i)) \\
&\quad + i \cdot (Im(\mathbf{K}_{i,j}) \cdot Re(\mathbf{x}_i) + Re(\mathbf{K}_{i,j}) \cdot Im(\mathbf{x}_i)).
\end{aligned} \tag{11.21}$$

If real and imaginary parts are represented as a vector, the following notation can be used:

$$\begin{bmatrix} Re(\mathbf{K}_{i,j}\mathbf{x}_i) \\ Im(\mathbf{K}_{i,j}\mathbf{x}_i) \end{bmatrix} = \begin{bmatrix} Re(\mathbf{K}_{i,j}) & -Im(\mathbf{K}_{i,j}) \\ Im(\mathbf{K}_{i,j}) & Re(\mathbf{K}_{i,j}) \end{bmatrix} \begin{bmatrix} Re(\mathbf{x}_i) \\ Im(\mathbf{x}_i) \end{bmatrix}. \tag{11.22}$$

The number of trainable parameters is determined by the kernel size k along each direction and the amount of input and output features $N_{f,in}$ and $N_{f,out}$. The number of trainable parameters in a convolution layer can be calculated as

$$\text{\#parameters} = N_{f,out} \cdot \left(1 + N_{f,in} \cdot \prod_p k_p \right), \tag{11.23}$$

where the kernel sizes k_p along each direction $p \in \{x, y, z, t\}$ are multiplied and where a bias term per filter is considered.

Box 11.4 Implementation of a convolution layer

A convolution layer is characterized by the following items:

- `in_channels` $N_{f,in}$: number of channels in the input image
- `out_channels` $N_{f,out}$: Number of output channels respectively feature maps in the result of the convolution, equals the number of filters to be trained
- `kernel_size` k : Size of the convolution kernel, integer, or tuple
- `padding` p : Boundary handling, impacts the image size, string, integer, or tuple
- `strides` s (optional): Spacing between pixel elements considered in the convolution, integer, or tuple
- `dilation_rate` d (optional): Spacing between kernel elements, integer, or tuple
- `bias` (optional): Adds a trainable offset to the output of the convolution, bool

A simple Keras 2D convolution layer can be created using

```
tf.keras.layers.Conv2D(out_channels, kernel_size, use_bias=True)
```

The input channels are automatically estimated in a separate `build` function.

All parameters need to be *initialized*. Commonly used initializers for filter kernels follow a Glorot Uniform [63] or a He Normal distribution [64]. The initializer can be passed directly to the Keras `Conv2D` by passing, e.g., `kernel_initializer="glorot_uniform"`.

If complex-valued input data are considered, two processing modes (see Section 11.4) are possible: 1) Real/Imaginary are considered in two input channels, hence, the output for each convolution filter is real-valued. However, the complex relationship between real and imaginary parts is lost; 2) Complex-valued convolutions as stated in Eq. (11.21) can be performed, where the output for each convolution filter is complex-valued. The drawback of this approach is that twice the number of trainable parameters is required as a real and imaginary filter kernel needs to be learned. Additionally, the number of convolution operations doubles compared to 1). In the context of MRI reconstruction, complex-valued processing is conducted both ways, as depicted in Table 11.1.

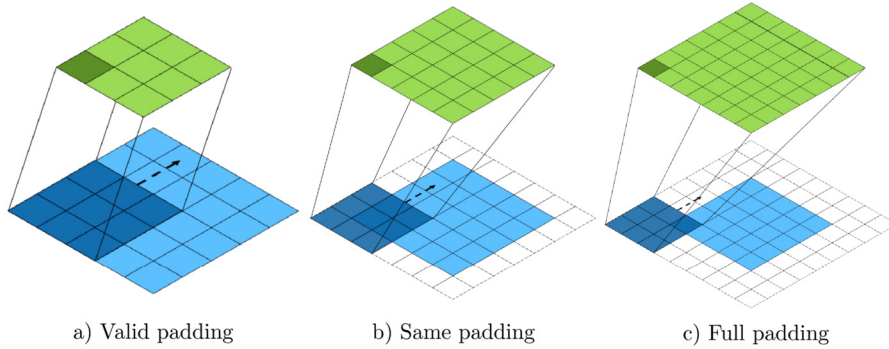


FIGURE 11.6 Convolution operation: Padding

Padding operations of the input. The input $\mathbf{x}$ (light blue (light gray in print version)) is padded with zeros (white) and then convolved with the filter kernel $\mathbf{k}$ (dark blue (dark gray in print version)) producing the output feature map (green (gray in print version)). The filter kernel of size 3×3 is slid over the entire input of size 5×5 yielding a scalar estimate (dark green (mid gray in print version)) at each position of the output feature map of size **a)** 3×3 for valid padding, i.e., $p = 0$, **b)** 5×5 for same padding with $p = 1$ in each direction, i.e., 1×1 padding so that output size is equal to input size, or **c)** 7×7 for full padding with $p = 2$ in each direction, i.e., 2×2 padding to enlarge output size. A stride of $s = 1$ and dilation rate $d = 1$ was used in all cases and all directions. Only a single filter is learned, i.e., output channel $N_{f,\text{out}} = 1$ from a single input channel $N_{f,\text{in}} = 1$.

Padding

Padding appends values at the boundary of the input, which for the majority of frameworks, is a zero-padding, i.e., a constant value of zero, but in general can follow periodic or symmetric padding as well. Three special cases for padding are of importance: 1) no or *valid padding* uses the input without padding ($p = 0$); 2) half or *same padding* appends zeroes so that the output has the same size as the input; and 3) *full padding* appends zeroes such that every pixel in the input contributes equally to the output, but the output size is enlarged. The number of padding p in these cases depends on the kernel size k and stride s . Fig. 11.6 illustrates these three special cases.

Stride

The *stride* describes the step size with which the filter kernel is moved over the input. A stride can be used to perform a down-sampling operation similar to a pooling layer (see Section 11.5.5). Fig. 11.7 shows a convolution operation with stride.

Receptive field

The receptive field describes the spatial extent of the input that a particular feature map within the network observes or is affected by. It is determined by the kernel_size k , padding p , stride s and dilation_rate d (see Section 11.5.1.1). These values are prescribed for each input direction and consequently influence the output size respectively receptive field. The output size along each dimension

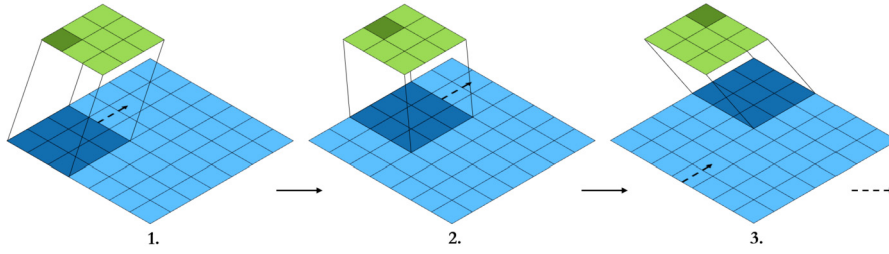


FIGURE 11.7 Convolution operation: Stride

Stride of the filter kernel moving over the input. The input $\mathbf{x}$ (light blue (light gray in print version)) is convolved with the filter kernel $\mathbf{k}$ (dark blue (dark gray in print version)) producing the output feature map (green (gray in print version)). The filter kernel of size 3×3 is slid over the entire input of size 7×7 with a stride of $s = 2$ yielding a scalar estimate (dark green (mid gray in print version)) at each of the output feature map of size 3×3 . No padding ($p = 0$) and a dilation rate $d = 1$ was applied in all directions. Only a single filter is learned, i.e., output channel $N_{f,\text{out}} = 1$ from a single input channel $N_{f,\text{in}} = 1$.

is for an input of size R_{in} given by

$$R_{\text{out}} = \left\lfloor \frac{R_{\text{in}} + 2p - d \cdot (k - 1) - 1}{s} \right\rfloor + 1. \quad (11.24)$$

For N -dimensional filters, these values can be stated as $R_{x,\text{out}} \times R_{y,\text{out}} \times \dots$. For an input $R_{x,\text{in}} \times R_{y,\text{in}} \times \dots \times N_{f,\text{in}}$, the convolution with $N_{f,\text{out}}$ filters of size $k_x \times k_y \times \dots$ (with padding $p_x \times p_y \times \dots$ and stride $s_x \times s_y \times \dots$) yields an output of size $R_{x,\text{out}} \times R_{y,\text{out}} \times \dots \times N_{f,\text{out}}$. For a single layer, the receptive field is the same as the size of the output from this layer. If several layers are applied sequentially, the receptive field can be retrieved by consecutive calculation of Eq. (11.24).

11.5.1.1 Dilated convolution

Dilated convolution, also known as atrous convolution, introduces a dilation rate along each direction to spread out the filter kernel. With this concept, the same receptive field can be achieved but with less trainable parameters. Fig. 11.8 depicts an example for a convolutional layer with dilation rate of $d = 1$, i.e., no dilation, and $d = 2$. In order to achieve the same receptive field, a filter kernel of 5×5 for $d = 1$ and of 3×3 for $d = 2$ can be used, yielding 25 ($d = 1$) versus 9 ($d = 2$) trainable parameters.

11.5.1.2 Separable convolution

While 2D and 3D implementations exist in ML frameworks, spatio-temporal and higher-dimensional convolutions might be designed in a different way. Separable convolution split the N -dimensional kernel operation into multiple steps. *Spatial separable convolution* splits the convolution along the spatial direction if they can be factored into independent directions. While N -dimensional convolutions can be used, various work have shown that separable convolutions can perform superior to N -dimensional convolutions [65,66]. Furthermore, less trainable parameters are required.

For example, in a separable 2D+t convolution, the 3D convolution kernel $\mathbf{k}$ along x , y and t can be separated into a 2D spatial and 1D temporal convolution filter if $\mathbf{k} = \mathbf{k}_{x,y} \otimes \mathbf{k}_t$ applies. Various other

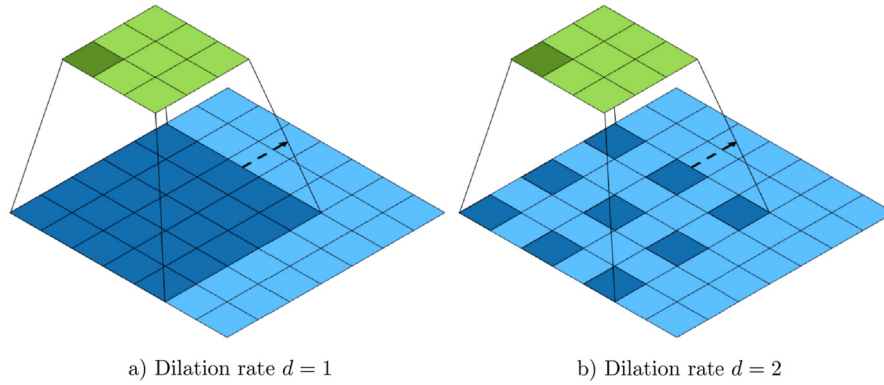


FIGURE 11.8 Dilated convolution

Dilated convolutions on the input $\mathbf{x}$ (light blue (light gray in print version)) which is convolved with the filter kernel $\mathbf{k}$ (dark blue (dark gray in print version)) producing the output feature map (green (gray in print version)). The filter kernel of size **a)** 5×5 for dilation rate of $d = 1$, i.e., no dilation, and **b)** 3×3 for a dilation rate of $d = 2$, i.e., dilation of 2×2 , is slid over the entire input of size 7×7 yielding a scalar estimate (dark green (mid gray in print version)) at each position of the output feature map of size 3×3 . A stride of $s = 1$ and valid padding $p = 0$ was used in all cases. Only a single filter is learned, i.e., output channel $N_{f,\text{out}} = 1$ from a single input channel $N_{f,\text{in}} = 1$.

variants exist to further improve the performance [67] or can be extended towards higher dimensions as shown in [66,68].

A *depthwise separable convolution* splits the spatial component from the depth/output channel component. An example calculation for the reduction in trainable parameters is shown in Appendix 11.C.

11.5.1.3 Transposed convolution

Transposed convolutions generally arise from the desire to use a transformation going in the opposite direction of a normal convolution, i.e., from something that has the shape of the output of some convolution to something that has the shape of its input, while maintaining a connectivity pattern that is compatible with convolutions. Transposed convolutions can thus be used for upsampling operations (see Section 11.5.6). Following Eq. (11.20), this yields the transposed filter $\mathbf{K}^T$ for $\hat{\mathbf{x}} = \mathbf{K}^T \mathbf{x}$.

11.5.2 Normalization layer

Adding normalization layers directly after convolution layers is a common way to enable faster and more stable training of NNs. Statistics are estimated from the input and used to reparametrize the input. Hence, the subsequent layers are less tolerant to changes in previous layers. The selection of the normalization layer is task dependent and there is no general recipe which layer should be selected. Although normalization layers are often important to make the NN train, they might lead to unwanted artifacts for image restoration tasks [69].

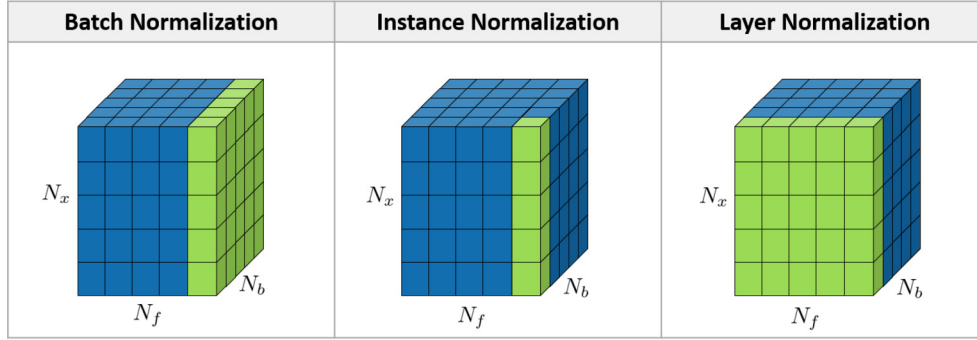


FIGURE 11.9 Normalization techniques

Visualization of common normalization techniques. The green (gray in print version) area denotes the dimensions that are used to calculate the normalization statistics, i.e., mean and standard deviation, over the input batch with dimensions $[N_b, N_x, N_f]$.

In this section, we define normalization layers n for the 1D case of a feature vector $\mathbf{x}$ with the dimensions $[N_b, N_x, N_f]$. We introduce the notation $E_{d=}\cdot[\mathbf{x}]$ and $Var_{d=}\cdot[\mathbf{x}]$ to compute the mean and variance values over the dimension $d \in \{b, x, f\}$. Common normalization methods are depicted in Fig. 11.9.

The normalization layer is defined as

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - E_d[\mathbf{x}]}{\sqrt{Var_d[\mathbf{x}] + \epsilon}} \alpha + \beta, \quad (11.25)$$

where $E[\mathbf{x}]$ and $Var[\mathbf{x}]$ are the estimated mean and variances of the input $\mathbf{x}$. The scaling parameter $\alpha \in \mathbb{R}$ and bias $\beta \in \mathbb{R}$ are optional parameters that can be learned.

Trabelsi et al. [70] introduced complex-valued batch normalization that ensures a circular distribution, i.e., the same variance for real and imaginary part. The scaling terms are replaced by covariance matrices, hence, complex batch-normalization reads as

$$\hat{\mathbf{x}} = \mathbf{A} \mathbf{V}^{-\frac{1}{2}} (\mathbf{x} - E_d[\mathbf{x}]) + \beta. \quad (11.26)$$

The matrix $\mathbf{V}$ defines the covariance matrix with real-valued components V_{rr} , V_{ii} and V_{ri} following

$$\mathbf{V} = \begin{bmatrix} E_d(Re(\mathbf{x})^2) & E_d(Re(\mathbf{x})Im(\mathbf{x})) \\ E_d(Im(\mathbf{x})Re(\mathbf{x})) & E_d(Im(\mathbf{x})^2) \end{bmatrix} = \begin{bmatrix} V_{rr} & V_{ri} \\ V_{ri} & V_{ii} \end{bmatrix}. \quad (11.27)$$

The scaling operator $\mathbf{A}$ contains (trainable) real-valued parameters α_{rr} , α_{ii} and α_{ri} following

$$\mathbf{A} = \begin{bmatrix} \alpha_{rr} & \alpha_{ri} \\ \alpha_{ri} & \alpha_{ii} \end{bmatrix}. \quad (11.28)$$

The bias $\beta \in \mathbb{C}$ is complex-valued.

Batch normalization

Ioffe et al. [71] introduced batch normalization to reduce the internal covariance shift. Statistics are computed over a mini-batch $d = b, x$, with batch size $N_b > 1$. During training, mini-batch statistics are accumulated over time. The accumulated running means $E[\bar{\mathbf{x}}]$ and variances $Var[\bar{\mathbf{x}}]$ are computed using a running average scheme with momentum ν

$$\begin{aligned} E[\bar{\mathbf{x}}] &= \nu \cdot E[\mathbf{x}] + (1 - \nu) \cdot E[\bar{\mathbf{x}}] \\ Var[\bar{\mathbf{x}}] &= \nu \cdot Var[\mathbf{x}] + (1 - \nu) \cdot Var[\bar{\mathbf{x}}], \end{aligned}$$

where $E[\mathbf{x}]$ and $Var[\mathbf{x}]$ are the estimated mean and variance of the current mini-batch $\mathbf{x}$. It follows that in training mode the running means and averages are only updated, while the real batch statistics are used to compute Eq. (11.25). In the inference mode, the running means and averages are used to compute Eq. (11.25).

Instance normalization

Initially introduced for style transfer, instance normalization is another widely used normalization technique for neural networks [72]. Here, $E[\mathbf{x}]$ and $Var[\mathbf{x}]$ are computed during training and tested individually for each sample and feature independently, i.e., $d = x$. The normalization is computed in the training and inference mode directly from the batch, hence, no moving average and moving variance have to be tracked during training. It is optional to learn additional scaling α and bias β parameters. The complex instance normalization follows Eq. (11.26).

Layer and group normalization

In contrast to batch normalization, *layer normalization* [73] computes the statistics over a whole layer, i.e., $d = x, f$. This is a common way to stabilize training of recurrent neural networks. *Group normalization* [74] divides the feature channels N_f into groups, and statistics are calculated within each individual group.

11.5.3 Activation layer

The activation layer decides if a neuron should be “fired” by analyzing the values of the convolution layer. This is realized by applying a nonlinear activation function in an element-wise way.

Rectified Linear Units (ReLU) are one of the most commonly used activation functions in ML. These piece-wise linear functions $\phi_{\text{ReLU}} : [-\infty, \infty] \rightarrow [0, \infty]$ set negative values to 0 and keep positive values following the rule

$$\phi_{\text{ReLU}}(\mathbf{x}) = \max(\mathbf{x}, 0).$$

Although ReLUs are not differentiable at $\mathbf{x} = 0$, which might cause issues in gradient-based learning, gradient-based optimizers use the subgradient for back-propagation and have shown to be efficient for a variety of ML tasks [14].

A common problem in training networks are *dying ReLUs* which output all zeros independent of the input values, making parts of the network inactive. To avoid this problem, *leaky ReLUs* are used

that apply a small gradient for negative values scaled by $\alpha < 0$, i.e.,

$$\phi_{\text{Leaky ReLU}}(\mathbf{x}) = \begin{cases} \mathbf{x} & \text{if } \mathbf{x} \geq 0 \\ \alpha \mathbf{x} & \text{if } \mathbf{x} < 0 \end{cases}.$$

This extends the output range $\phi_{\text{Leaky ReLU}} : [-\infty, \infty] \rightarrow [-\infty, \infty]$. A further extension is *parametrized ReLUs (PReLU)* that allow the scale α to be learned for the individual feature channels.

When applying nonlinear activation functions ϕ to complex values, several things have to be considered. One possibility is to apply the activation function to the real and imaginary part separately, however, the natural correlation between real and imaginary channels are not considered in this case. Furthermore, the phase information is mapped to the first quadrant, i.e., the interval $[0, \frac{\pi}{2}]$. Another variant is to keep the phase information fixed and only alter the magnitude information. An example is the *ModReLU* [75]

$$\phi_{\text{ModReLU}}(\mathbf{x}) = \max(0, |\mathbf{x}| + \beta) \frac{\mathbf{x}}{|\mathbf{x}|},$$

where β is the bias that is trainable.

Virtue et al. [76] proposed a new complex activation function called *cardioid*

$$\phi_{\text{cardioid}}(\mathbf{x}) = \frac{1}{2} (1 + \cos(\angle \mathbf{x} + \beta)) \mathbf{x}.$$

The complex cardioid can be seen as a generalization of ReLU activation functions to the complex plane. Compared to other complex activation functions, the complex cardioid acts on the input phase rather than the input magnitude. A bias β can be additionally learned.

11.5.4 Fully connected layer

Fully connected (FC) (or dense) layers connect every input unit to every output unit and are formulated as dense matrix–vector multiplication so

$$\hat{\mathbf{x}} = \mathbf{W}\mathbf{x} + \beta, \tag{11.29}$$

with an additional bias β . Trainable parameters are the dense matrix $\mathbf{W}$ and the bias β . FC layers are global feature extractors and are core components in multi-layer perceptrons (MLPs), object recognition, classification, and global transform learning [46]. In contrast to convolution layers, the number of trainable parameters depend on the amount of input nodes N_i and output nodes N_o (including bias β), yielding $\text{\#parameters} = N_i \cdot N_o + N_o$. This becomes inefficient to compute when learning global transforms of large image sizes.

11.5.5 Down-sampling layer

Down sampling layers reduce the spatial resolution and make NNs more efficient to train as the number of operations are reduced substantially in subsequent layers. *Pooling layers* are used as down-sampling operations to reduce the spatial resolution in the image and to introduce approximate invariance to

small translations [14]. Small patches of size k are analyzed in the individual features maps to keep important information about extracted features. Common pooling layers are *average pooling* and *max pooling*. For complex-valued images, the maximum operation does not exist. Instead, the pooling layer is modified such that it keeps values with, e.g., the maximum magnitude response. A pooling layer does not have any trainable parameters.

Convolutional Downsampling layers are convolution layers (see Section 11.5.1) with a stride greater than 1, resulting in an output with decreased spatial resolution. Down sampling with convolution layers can be seen as a learning the pooling operation. However, the number of trainable parameters are increased when using convolutional downsampling.

11.5.6 Up-sampling layer

Up-sampling layers increase the spatial resolution and are important to revert from coarser scales to the original image resolution in image-to-image networks. *Convolutional Upsampling* with transposed convolutions (see Section 11.5.1) allow to recover the original input size, for strides $s > 1$. However, this kind of upsampling is known to introduce checkerboard artifacts [77]. The number of trainable parameters increases with convolutional upsampling. To overcome checkerboard artifacts in convolutional upsampling, nearest neighbor or bilinear *interpolation* provide an alternative. Interpolation is commonly followed by a convolution layer for further computation. Bilinear interpolation in the complex domain can be conducted by either interpolating real/imaginary part or magnitude/phase separately.

11.5.7 Dropout layer

Dropout [78] is a regularization technique during training to prevent overfitting of the network. At each training iteration, a random selection of nodes and connections to and from the layer are temporarily set to zero with probability $1 - p$ or kept with probability p . The result of dropout is scaled with a factor $1/p$ to maintain the scale of the output layer. The complex dropout is realized the same way to the complex number, but not to real and imaginary parts separately.

11.5.8 Merging layers

Concatenation stacks tensors together along a certain dimension, mostly the feature dimension. The aim is to increase feature diversity or to gather feature information from multiple parts of the network. It is also possible to merge further inputs into the network at deeper layers. Note that the spatial dimensions have to match when using merging layers at various scales. If concatenation is followed by a convolution layer, this can be interpreted as weighted combination of multiple inputs. In a similar sense, other feature map combinations, such as *addition*, *subtraction*, *multiplication*, *averaging*, *minimum* or *maximum*, can be performed.

11.5.9 Recursive layer

Recursive layers enable processing of spatiotemporal sequences of arbitrary length. The temporal frames are processed sequentially, but the temporal features are captured and stored in hidden units

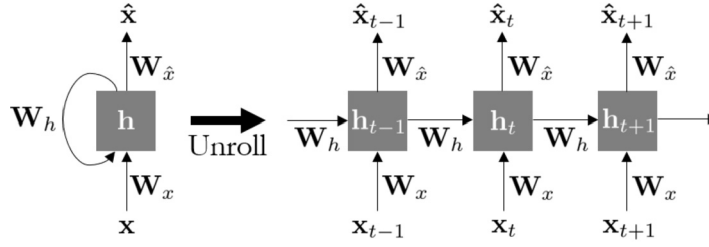


FIGURE 11.10 Recursive layer

Recursive layer with input $\mathbf{x}$, hidden unit $\mathbf{h}$, and output $\hat{\mathbf{x}}$ to process a sequence of time frames t . Inputs and outputs are weighted by learned filters $\mathbf{W}_x$ and $\mathbf{W}_{\hat{x}}$, respectively, as well as the recursive feedback $\mathbf{W}_h$. Unrolling the recursive unit reveals its temporally sequential processing.

that act as memory. Later frames have access to this memory. A basic recursive layer reads as

$$\begin{aligned} \mathbf{h}_t &= \phi_h(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b_x) \\ \hat{\mathbf{x}}_t &= \phi_{\hat{x}}(\mathbf{W}_{\hat{x}} \mathbf{h}_t + b_{\hat{x}}), \end{aligned} \quad (11.30)$$

where: $\mathbf{W}_h$, $\mathbf{W}_x$, and $\mathbf{W}_{\hat{x}}$ are weight matrices; b_x and $b_{\hat{x}}$ are the bias; ϕ_h and $\phi_{\hat{x}}$ are the activation functions; and $\mathbf{h}$ are the hidden units receiving input $\mathbf{x}$ and output $\hat{\mathbf{x}}$ for the processed temporal frame t . Fig. 11.10 depicts the recursive layer together with its unrolled version in which the temporally sequential processing can be appreciated. The formulation in Eq. (11.30) might result in vanishing gradients during back propagation as each step recurrently depends on all previous steps. This might be avoided by introducing additional *gates* such as in Long Short Term Memory (LSTMs) [6] and Gated Recurrent Units (GRUs) [79]. Note that the weight matrices are defined by convolutions in the case of image reconstruction tasks. While Eq. (11.30) can be straightforwardly translated to complex-valued operations, LSTMs and GRUs require real-valued gating functions as defined in, e.g., [80].

11.5.10 Building blocks

Several layers can be bundled to form larger building blocks as shown in Fig. 11.11. *Convolution blocks* in NNs commonly consist of a sequence of convolution, normalization (optional) and activation. *Residual blocks* [81] learn a residual that is added to the input of the block. The direct identity connections are also known as skip or residual connections. Furthermore, this avoids the vanishing gradient problem and makes the network easier to train. Depending on the kernel sizes and channels, these residual blocks can be also referred to as *bottleneck residuals* (including 1×1 convolutions) or *wide residuals* (increased channel depth) [82]. *Dense blocks* [83] follow a similar principle, but they concatenate feature maps from several previous layers, determined by the growth rate g , to build a collective knowledge. It is also possible to train a convolution filter for the residual/dense path which acts then as a weighted combination or attention focusing depending on if also acting on spatial dimension, also known as *fire blocks* [84], or spatial/channel squeezing/excitation, known as *squeeze-and-excite* [85]. Several convolution blocks acting in parallel to a residual connection form an *assembly block*. An alternative assembling can be achieved with *merge-and-run mapping* [86] which averages the inputs of the

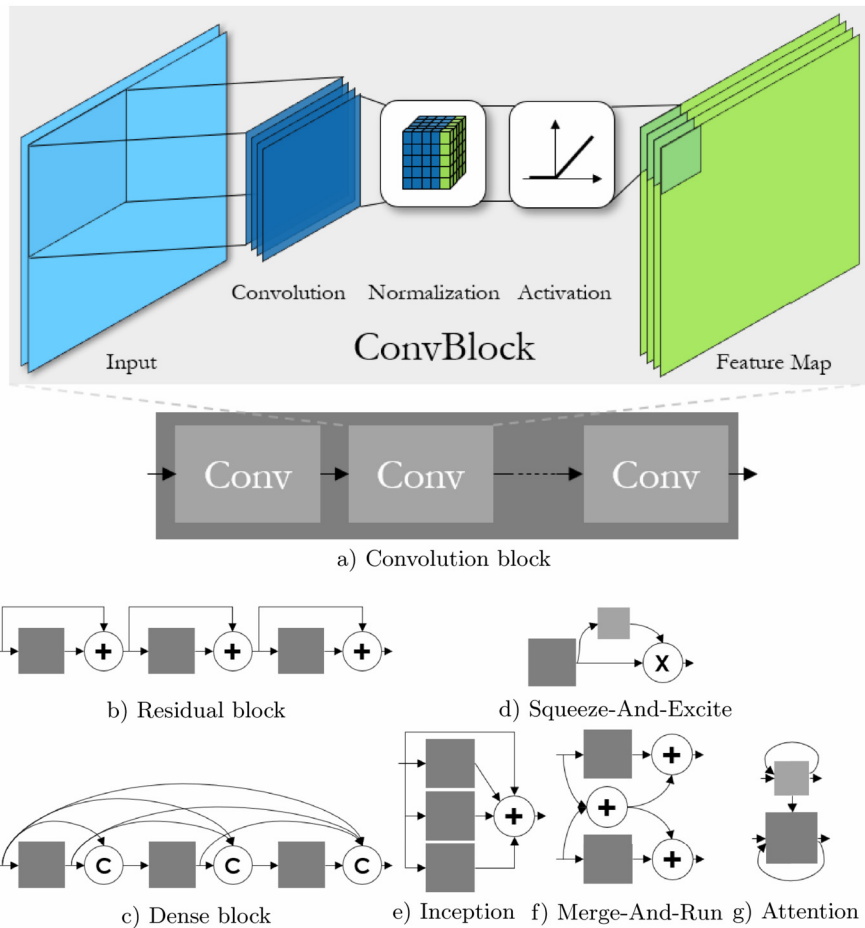


FIGURE 11.11 Building blocks

Common building blocks in neural networks. **a)** Convolution blocks are used for processing image input via convolution layer, followed by normalization (optional) and activation function to form the feature-map output. Several consecutive convolution blocks are usually taken inside other building blocks. **b)** Residual block with residual/skip connection and addition of feature map at output. **c)** Dense block with skip connections over several stages and concatenation of feature maps. **d)** Squeeze-and-excitation by learning a spatial/channel weighting to scale the output feature map. **e)** Inception with multiple parallel convolution blocks of various kernel sizes to allow for a multiresolution approach. **f)** Merge-and-run mapping for merging information between parallel branches. **g)** Attention mechanisms via recursive layers to focus the layer output.

residual branches (merge) and adds the average to the output of each residual branch as the input of the subsequent residual branch (run). A multiresolutional approach is achieved if convolution layers/blocks

of different kernel sizes are operating in parallel, known as *inception blocks* [87]. Several versions of this have been proposed [88,89].

Attention mechanisms originated from the field of natural language processing and help to learn dependencies between source and target, consequently focusing the network's attention on the most significant information. The methods differ in the used alignment scores (content [90], additive [91], location [92] or scaled dot-product [93]) and information sharing (self-attention [94], global/soft [95], and local/hard [92,95]). Within these concepts, *recurrent Neural Networks (RNNs)* (see Section 11.5.9) are used to extract the sequence of information based on their relative temporal spreading.

RNNs thus form another type of building blocks that are especially advantageous to process spatiotemporal sequences of arbitrary length. The temporal frames are processed sequentially with temporal information stored in hidden units which later frames can access. Applications of RNNs for MRI reconstruction can be found in, e.g., [96].

11.5.11 Data consistency layers

DC layers are the core layers for physics-based learning Section 11.4.3 and are responsible to induce similarity to the acquired k -space data $\mathbf{s}$. A DC layer receives as input the currently reconstructed image $\mathbf{x}$, its corresponding k -space $\mathbf{s}$, and information about the sampling pattern, e.g., a sampling mask in the Cartesian case or a sampling trajectory in the non-Cartesian case (Chapter 4). The implementation of the DC layers follows the definitions of the encoding operator $\mathbf{E}$. The implementation of various DC layers follows directly the mathematical derivation. In the following, we focus on the most commonly used DC layers that impose DC by gradient descent [97], null-space projection [98], or proximal mapping [51,96,50]. For details on other realizations of DC layers, such as variable splitting and primal-dual optimization, we refer the interested reader to the original publications [52,53].

Gradient descent (GD-DC) layer

Let us consider a GD scheme presented in Eq. (11.15). This involves the encoding operator $\mathbf{E}$ and its adjoint operator $\mathbf{E}^H$. In the Cartesian single-coil case, the forward operator is defined as $\mathbf{E} = \mathbf{B}\mathbf{F}$, and its adjoint is defined as $\mathbf{E}^H = \mathbf{F}^H\mathbf{B}^H$. Since the sampling mask $\mathbf{B}$ is a diagonal matrix consisting of ones and zeros, it follows $\mathbf{B}^H = \mathbf{B}$. The adjoint FT equals the inverse FT, i.e., $\mathbf{F}^H = \mathbf{F}^{-1}$. Using these definitions, the GD-DC layer can be easily implemented as

$$\hat{\mathbf{x}} = \mathbf{x} - \lambda \mathbf{E}^H(\mathbf{E}\mathbf{x} - \mathbf{s}), \quad (11.31)$$

where $\lambda > 0$ impacts the weight of DC and is fixed or trainable. This scheme works also for the multicoil as presented in [97].

Null-space projection (NSP-DC) layer

Mardani et al. [98] introduced a layer to project onto the nullspace of $\mathbf{E}$. The NSP-DC layer reads as

$$\hat{\mathbf{x}} = \mathbf{E}^\dagger \mathbf{s} + (\mathbf{I} - \mathbf{E}^\dagger \mathbf{E})\mathbf{x}, \quad (11.32)$$

where $\mathbf{E}^\dagger$ denotes the pseudo inverse.

Proximal mapping (PM-DC) layer

This DC layer is shaped by computing the proximal mapping from Eq. (11.17). For single-coil PM-DC, this yields

$$\hat{\mathbf{x}} = \mathbf{F}^H \mathbf{\Lambda} \mathbf{F} \mathbf{x} + \frac{\lambda}{1 + \lambda} \mathbf{F}^H \mathbf{B}^H \mathbf{s}, \quad \mathbf{\Lambda}_{ii} = \begin{cases} 1 & \text{if } i \notin \Omega \\ \frac{1}{1 + \lambda} & \text{if } i \in \Omega, \end{cases} \quad (11.33)$$

where $\mathbf{\Lambda}$ is a diagonal matrix and Ω defines the index set of acquired k -space samples. For the multicoil, no closed-form solution to Eq. (11.17) exists, and Aggarwal et al. proposed to solve this problem using a conjugate gradient (CG) optimization. Therefore, Eq. (11.17) is computed and rearranged to form $\mathbf{A} \mathbf{x} = \mathbf{b}$, to get the system matrix $\mathbf{A}$ and the right-hand side term $\mathbf{b}$ for the CG optimizer.

11.6 Network architectures for MRI reconstruction

All the presented layers and building blocks can be used to form a full network. For ML-based MR reconstructions, most commonly CNNs are used that are a sequence of convolution blocks with the aforementioned interconnections. There is, however, no unique definition of networks, and numerous variants exist, depending on the underlying task [45,99–102]. The targeted application, dimensionality, and data availability mainly determine the task definitions (see Section 11.4) and, subsequently, the architectural choices. An overview of various recent network architectures for MRI reconstruction with distinction by task definition are stated in Table 11.1.

A UNet [103] processes the input at various scales, involving convolution blocks, global and local skip connections, down sampling, up sampling and merging layers. Various variants for MRI reconstruction can be found in, e.g., [66,100,104]. ResNet [81] form a large backbone of existing reconstruction networks [51,96,105]. In Variational Networks (VNs) [97,106,107], the main building blocks are motivated by fields-of-experts regularization [108] and have a specific structure defined by the gradient of an energy model. This also involves trainable activation functions, realized with, e.g., Gaussian radial basis functions, linear interpolation, or spline interpolation. Generative adversarial networks (GANs) have been studied to model human-perceived image quality or quality control within the reconstruction process by the adversarial training of generator and discriminator [44,109,110]. Reconstructions can be formulated, as unrolled optimizations [51,66,96,97,111], direct mappings [46], k -space interpolations [61], plug-and-play priors [60,112], or super-resolution task [113], depending on the sampling trajectory and application. In Fig. 11.12, three networks with their targeted application of cardiac, neuro, and MSK imaging are depicted.

11.7 How to build an ML model for MR reconstruction

In this section, we provide an overview of the fundamental study and design choices when building a ML model for MR reconstruction.

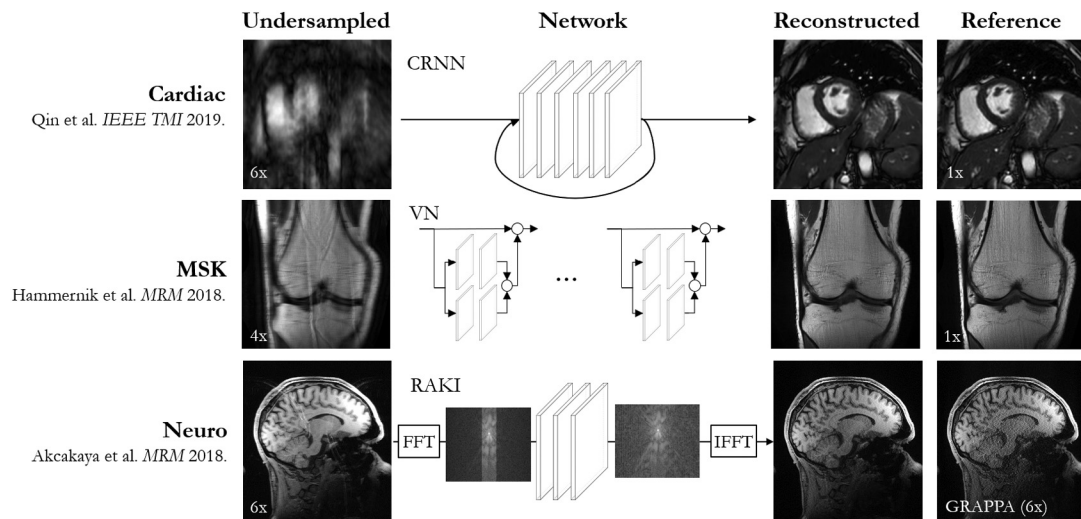


FIGURE 11.12 Machine Learning for MRI reconstruction

Three applications of cardiac, neuro, and musculoskeletal imaging with their proposed machine learning networks for image reconstruction. Details of the networks can be found in Table 11.1.

11.7.1 Checklist to build an ML model

In Table 11.2, we provide an example checklist of important and interdependent points that need to be considered to decide on and design an appropriate architecture.

11.7.2 Database

For training an ML method, a database is required that provides sufficient diversity, but with substantial consistency among datasets targeted to the underlying application. The amount of data required to train an application depends on the complexity of the problem, the complexity of the learning algorithm, and the expected level of performance.

When confronted with a database, you might consider the following:

- Does the database actually resemble the problem?
- Is your database suitable for self-supervised or supervised learning schemes (see Section 11.3.2)?
- Can you use the database as a pre-training step and fine-tune on more specified data?
- Can you collect/acquire more (task-specific) data?
- Can you use data augmentation to increase sample size?
- Can you use phantom data (numerical or acquired) or natural images to mimic the application?

Acquiring new data samples for ML-based MR reconstructions is often not trivial because it requires acquisition of fully sampled (or at least aliasing-free) ground-truth data. Furthermore, simply increasing the sample size does not guarantee a better performance [120,121].

Table 11.1 Network Architectures for MRI Reconstruction. Comparison of network architectures for MRI reconstruction in relation to task definitions (see Section 11.4), processing dimensionality, and input. The processing dimensionality also considers how temporal information (if available) is handled (see Appendix 11.A and Section 11.5.1). Multicoil indicates that information from several MR receiver coils via coil sensitivities were considered inside the network or data consistency. Methods either deploy real-valued processing considering the complex nature of the data in two channels or two networks or perform complex operations (see Appendix 11.A). The input indicates which data (mag: magnitude, pha: phase, real: real, imag: imaginary) is sent into the denoising network (*) deployed in the underlying task.

Task Definition	Method	Dimensionality	Single-/Multicoil?	Real(R)-/Complex(C)-valued?	Input*
Image denoising	Deep Residual [114]	2D	single-coil	R, 2 networks	image (mag/pha)
Image denoising	DAGAN [44]	2D	single-coil	R, 1 channel	image (real)
Image denoising	GANCS [109]	2D	single-coil	R, 1 channel	image (mag)
Image denoising	CNN-SR [113]	2D	single-coil	R, 1 channel	image (mag)
Image denoising	SANTIS [110]	2D	multicoil	R, 2 channels	image (real/imag)
Image denoising	Residual learning [43]	3D	single-coil	R, 1 channel	image (magnitude)
Direct mapping	AUTOMAP [46]	2D	single-coil	R, 2 channels	k -space (real/imag)
PnP	[115]	2D	single-coil	R, 2 channels	image (real/imag)
PnP	RARE [112]	3D	single-coil	R, 2 channels	image (real/imag)
PnP	PnP-CNN [60]	3D	multicoil	R, 2 channels	image (real/imag)
Hybrid	KIKI-net [62]	2D	single-coil	R, 2 channels	image and k -space (real/imag)
k -space learning	ALOHA [116]	2D	single-coil	C	k -space (real/imag)
k -space learning	RAKI [61]	2D	multicoil	R, 2 channels	k -space (real/imag)
k -space learning	sRAKI [117]	2D	multicoil	R, 2 channels	k -space (real/imag)
k -space learning	DeepSpirit [118]	2D	multicoil	R, 2 channels	k -space (real/imag)
Unrolled optimization	ADMM-Net [119]	2D	single-coil	C	k -space (real/imag)
Unrolled optimization	MoDL [111]	2D	multicoil	R, 2 channels	image (real/imag)
Unrolled optimization	DeepCascade [51]	2D+t	single-coil	C	image (real/imag)
Unrolled optimization	CRNN [96]	2D+t	single-coil	C	image (real/imag)
Unrolled optimization	VN [97]	2D	multicoil	C	image (real/imag)
Unrolled optimization	VN [106]	2D	multicoil	C	image (real/imag)
Unrolled optimization	VN [107]	2D	multicoil	C	image (mag/pha)
Unrolled optimization	DL-ESPIRiT [105]	2D+t	multicoil	C	image (real/imag)
Unrolled optimization	CINENet [66]	3D+t	multicoil	C	image (real/imag)

Publicly available databases are rare in the field of medical imaging because of privacy, data confidentiality, and ethical considerations. However, some funding sources request/provide access to the data used in scientific studies (IXI [122], ADNI [123], OASIS [124], etc.) and an increasing trend towards data sharing can be observed (e.g., OpenfMRI [125]). Some databases exist in the form of challenges (fastMRI [126], CHAOS [127], etc.) that were hosted at conferences (MICCAI, ISBI, ISMRM, etc.) and are summarized here: <http://grand-challenge.org>.

Another challenge in medical imaging is to define the ground-truth in the case of supervised reconstruction approaches. Ideally, fully sampled data serves as reference from which in a retrospective undersampling, input data is created that has a 1:1 voxel-wise correspondence to the ground truth and, hence, enables the usage of voxel-wise loss functions for optimization. However, ground-truth data can

Table 11.2 Checklist for building an ML model. Checklist for designing a learning-based MRI reconstruction architecture.
Targeted application and task definition (see Section 11.4)
☐ Unrolled MRI reconstruction for 2D/3D static/dynamic imaging ☐ Post-processing of pre-reconstructed images ☐ k -space processing ☐ Hybrid reconstruction ☐ Plug'n'Play Prior ☐ other:
Data availability
☐ Acquired or simulated measurement data ☐ Pre-processed data ☐ Online database ☐ Existence of ground-truth data, i.e., type of learning, see Section 11.3.2
Data handling (see Appendix 11.A)
☐ Data format: ☐ image and/or ☐ rawdata ☐ Data type: ☐ real-valued or ☐ complex-valued ☐ Data dimensionality: ☐ 2D, ☐ 2D ☐ 2D+t, ☐ 3D, ☐ 3D+t, ☐ ND ☐ Pre-processing ☐ other:
Data splitting
☐ Creation of training, validation and test set ☐ Cross-validation needed
Model design
☐ Pre-trained or similar conceptual model available, see Section 11.6 ☐ Consistency to measured k -space required, see Section 11.5.11 ☐ Complex-valued handling, see Appendix 11.B ☐ Single- or multicoil handling, see Section 11.4 ☐ Temporal or high-dimensional data handling, see Appendix 11.A ☐ Network architecture and building blocks, see Section 11.5.10 ☐ Network input: ☐ corrupted image, ☐ k -space, ☐ sampling masks / trajectories ☐ coil-sensitivity maps, ☐ other ☐ Network output: ☐ reconstructed image, ☐ reconstructed k -space
Network training
☐ Losses, see Section 11.3.3 ☐ Optimizer, see Section 11.3.3 ☐ Monitoring of training and hyperparameter optimization, see Section 11.3.4
Evaluation
☐ Quantitative evaluation: ☐ PSNR, ☐ SSIM, ☐ MSE, ☐ other ☐ Qualitative evaluation: ☐ Images, ☐ difference images, ☐ blinded expert reading, ☐ other

in some cases be challenging or impossible to acquire. Even fully sampled data might not be ideal and may suffer from measurement noise and additional artifacts due to the longer acquisition. Additionally, clinical studies to collect patient data are expensive and time consuming. If feasible, phantom data (numerical or acquired), *data augmentation*, or *transfer learning* from other databases can be conducted, also helping to cope with small sample sizes, as detailed in the following.

Pre-training and transfer learning

Transfer learning describes the process of pre-training a network on a dataset with a large amount of samples and then *transferring* and fine-tuning the trained model parameters on a specific tasks where fewer data samples are available. However, pre-trained models for medical images, especially for complex-valued MRI reconstruction, so far don't exist, and leveraging models trained on natural scene images [128] can only be performed to a certain extent, e.g., for perceptual losses or discriminator networks that are applied to the magnitude image. For MRI reconstruction, transfer learning has been used for domain adaptation from CT to radial MRI reconstruction [104] or between imaging sequences [129]. Transfer learning on natural images serves as training database in [46,130,131]. Synthetic phase information, e.g., sinusoidal phase at random spatial frequencies, is added to simulate the complex domain. Multicoil data might be simulated with coil sensitivity maps estimated from real MRI scans.

Data augmentation

Data augmentation is another popular pre-processing step to generate a more versatile dataset. In general, transformations such as rotation, translation, shearing, scaling and nonrigid transformations are used. However, experience from natural image processing cannot be transferred easily to obtain realistic data augmentation for MRI reconstruction. Realistic data augmentation require physics-aware transformations. This can be realized by, e.g., simulating motion [132] or augmentation of the subsampling pattern (beyond changing acceleration factors). Realistic adversarial data augmentation has been introduced, particularly for MRI segmentation, so far [133].

11.7.3 Database pipeline

Data loading pipelines are an essential and integral part of the development. Pipelines are usually specific to the underlying data source, layout, application, and selected ML framework. Some efforts have been made to unify processing in training and testing such as NiftyNet [134] for Tensorflow or TorchIO [135] for PyTorch. Some considerations need to be taken with respect to data loading and format: Data can be pre-processed and stored in an efficient (containerized) format (e.g. TFRecords, HDF5) to enable fast loading of samples into framework during training. The order of the tensor dimensions in the stored file and how they are processed/accessed within the framework also needs to be considered to minimize extensive reshaping operations. Depending on the application, further pre-processing steps (scaling, intensive augmentation, etc.) can be performed as well to minimize the workload during training. If a smaller sample size is used, the database may even be loaded directly into the RAM, instead of being fetched from the hard drive. If fetching from the hard drive is conducted, pre-fetching of data can be considered, e.g., the pipeline loads batches of samples ahead (operated on CPU) while training on previously loaded ones (operated on GPU). This operation can be parallelized and improves overall performance. Core functionalities for this are provided in the respective frameworks.

For testing, access to the trained ML infrastructures through plugins connected to a graphical user interface such as DeepInfer [136] for Slicer3D [137], NiftyLink for NIFTK [138], NvidiaAnnotation-Plugin for MITK [139], or Nora [140] have been established. They facilitate the labeling and inspection of test results.

11.7.4 Frameworks

Building your own ML application from scratch is hardly possible nowadays. There exist a number of frameworks that support tremendously the development and production of ML applications. These frameworks take care of core functionalities, automatic differentiation, have CPU/GPU support, provide the user with a numerous amount of tools to process data, build models, and train/evaluate the models to make the start into ML easier. These frameworks are open-source and can be used with python.

- TensorFlow³ is a ML library developed by Google. Complex-valued support is available. The core of TensorFlow is the computation of graphs. Graphs consist of nodes, describing operations, and edges, which are tensors. The graphs need to be *compiled* before training/evaluation the ML model. While these static graphs have several advantages, e.g., computational optimization and distributed computing, debugging is cumbersome if the eager mode is disabled.
- Keras⁴ is a simple API that builds on top of Tensorflow (and Theano). It encapsulates a number of steps in the development of ML algorithms such that only minimal interaction and coding is required from the user.
- PyTorch,⁵ developed by Facebook Inc. Pytorch, is a more lightweight ML library, and intuitive to use for Python/NumPy users. Graphs are built dynamically and default execution mode is eager, which allows for easy debugging. PyTorch offers checkpoint processing, which allows for memory-efficient training. Complex-valued support is available for recent versions.

11.8 Summary

This chapter introduced the basic concepts of performing MR image reconstruction with ML frameworks. First, the fundamentals of ML were discussed, including types of learning, cost function, optimization, training/validation/test set composition and their usage. The underlying MR application, including applied body region, imaging sequence, data dimensionality, etc., determine the MR reconstruction task. The various reconstruction tasks differ in their input and targeted application output. Image denoising, direct mapping, and physics-based reconstructions have been introduced and their advantages and disadvantages discussed. Basic building blocks form the backbone of the reconstruction networks and are composed depending on the task and application specific reconstruction. Exemplary ML networks for MR image reconstructions were shown and discussed. A checklist was provided that can guide the initial setup of ML reconstruction frameworks. The chapter concludes with examples and tutorial codes for practice.

11.9 Further resources and tutorials

In the following, a list of resources for further reading is provided. This (incomplete) list includes online resources, tutorials, books, book chapters and survey papers.

³ <https://www.tensorflow.org/>.

⁴ <https://keras.io/>.

⁵ <https://pytorch.org/>.

- Keras: <https://keras.io/>
- Tensorflow tutorials: <https://www.tensorflow.org/tutorials>
- Tensorflow playground: <https://playground.tensorflow.org>
- PyTorch tutorials: <https://pytorch.org/tutorials>
- Peter Chang's DL tutorials: https://github.com/peterchang77/dl_tutor
- Neural network zoo: <https://www.asimovinstitute.org/neural-network-zoo/>
- Interactive deep learning tutorial: <https://d2l.ai/>
- Udacity AI courses: <https://www.udacity.com/courses/school-of-ai>
- Bishop, *Pattern Recognition and Machine Learning* [13]
- Vapnik, *The Nature of Statistical Learning Theory* [120]
- Murphy, *Machine Learning: A Probabilistic Perspective* [15]
- Goodfellow et al., *Deep Learning* [14]
- Survey papers [99, 141–143]

11.10 Exercises

11.1 Derive the proximal mapping for single-coil reconstruction

$$\text{prox}_{\tau g}(\hat{\mathbf{x}}) = \arg \min_{\mathbf{x} \in \mathbb{K}^n} \frac{1}{2\tau} \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + g(\mathbf{x}),$$

where

$$g(x) = \frac{\lambda}{2} \|\mathbf{B}\mathbf{F}\mathbf{x} - \mathbf{s}\|_2^2.$$

11.2 No closed-form solution of the proximal mapping exist for multicoil reconstruction

$$\text{prox}_{\tau g}(\hat{\mathbf{x}}) = \arg \min_{\mathbf{x} \in \mathbb{K}^n} \frac{1}{2\tau} \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + g(\mathbf{x}),$$

where

$$g(x) = \frac{\lambda}{2} \|\mathbf{E}\mathbf{x} - \mathbf{s}\|_2^2.$$

A CG optimization can be used to solve this problem. First, derive the equation that has to be solved using CG in the form $\mathbf{A}\hat{\mathbf{x}} = \mathbf{x}$. Second, derive the equation to back-propagate the error $\mathbf{e} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}}$ with respect to the input $\hat{\mathbf{x}}$. How can the error be back propagated with CG practically?

Hint. Use following rule to compute the derivative of the matrix inverse $\mathbf{X}^{-1}$

$$\partial \mathbf{X}^{-1} = -\mathbf{X}^{-1} (\partial \mathbf{X}) \mathbf{X}^{-1}.$$

11.3 Implement a 3D convolution layer with 1 input channel and 32 output channels in the complex domain and ModReLU activation functions. First, use 3D convolutions with a filter size of $5 \times 5 \times 3$. How many trainable parameters does this layer have? Second, design a 2D+t convolution layer

that uses $5 \times 5 \times 1$ spatial filters, followed by $1 \times 1 \times 3$ temporal filters. How many intermediate features, i.e., output channels of the spatial convolutions do you need to reach the same model complexity as the 3D convolution?

- 11.4** Calculate the number of network parameters and the receptive field for a 5-layer CNN with, 64 features in the first 4 layers, and one output feature in the last layer. The kernel size is set to 3, bias is applied, and ReLU activations are used. The input is a single-channel real-valued image. How does the number of parameters change if this is performed in the complex-valued domain, using complex-valued convolutions and cReLU activations?
- 11.5** Global transform learning as presented in Zhu et al. [46] is based on fully connected layers. Assume, you have a 64×64 input image, yielding a $N = 64^2$ input. You build a network as follows: FC layer with N hidden neurons, followed by a FC layer with N hidden neurons. This output is reshaped to $n \times n$ and followed by two convolution layers with 32 feature maps. The final convolution maps to a single-channel output image. Compute the number of parameters for the fully complex-valued case and the 2-channel real case. How does the number of parameters change for a 64×64 , 128×128 and 512×512 , assuming the single-coil case?

Hint. For the 2-channel real case, the first FC layer has $2N$ neurons.

11.10.1 Hands-on examples

The following four exercises correspond to hands-on examples to experiment with basic building blocks, data consistency layers, and network architectures on toy examples in Tensorflow/Keras. Material is provided online,⁶ where an overview of the tutorials is given in `merlin.ipynb`. The examples are run on the MNIST [144] dataset with complex-valued data and k -space data being simulated. All examples are provided as Google Colab notebooks,⁷ which allow the user to access GPUs to run the code with no payment.

- 11.6** Image-denoising based MR reconstruction on real-valued images:
This example performs a basic CNN and a residual CNN for an image denoising MR reconstruction task on real-valued images. Follow instructions and steps in `tutorial_denoising_real.ipynb` for this example.
- 11.7** Image-denoising based MR reconstruction on complex-valued images with complex-valued processing:
This example performs a basic CNN and a residual CNN for an image denoising MR reconstruction task on complex-valued images with complex-valued processing. Follow instructions and steps in `tutorial_denoising_complex.ipynb` for this example.
- 11.8** Image-denoising based MR reconstruction on complex-valued images with real-valued processing:
This example performs a basic CNN and residual CNN for an image denoising task on complex-valued images with real-valued processing as complex data sent in as 2-channel input. Follow instructions and steps in `tutorial_denoising_2chreal.ipynb` for this example.

⁶ <https://github.com/midas-tum/merlin/blob/master/notebooks/>.

⁷ <https://colab.research.google.com/>.

11.9 Physics-based MR reconstruction on complex-valued images:

This example performs an unrolled reconstruction network with denoising regularizer and data consistency blocks for complex-valued input. Follow instructions and steps in `tutorial_physicsbased_complex.ipynb` for this example.

Appendix 11.A ML-specific notation

The acquired k -space data $\mathbf{s} \in \mathbb{C}^K$ are a complex-valued signal usually acquired with multiple receiver coils. Here, we use a common notation for the input signal $\mathbf{x} \in \mathbb{K}^N$ (image or k -space, depending on the task definition Section 11.4) to represent both real and complex signals where $\mathbb{K}$ denotes a set. In the real-valued case, $\mathbf{x} \in \mathbb{R}^N$ is element of the real-valued numbers $\mathbb{R}$. In the complex-valued case, $\mathbf{x} \in \mathbb{C}^N$ is element of the complex-valued numbers $\mathbb{C}$ and can be represented by its real and imaginary components as $\mathbf{x} = \text{Re}(\mathbf{x}) + i\text{Im}(\mathbf{x})$.

Dimensionality

N denotes the dimensionality of the image $\mathbf{x}$ and is defined by the product of its image dimensions. In ML frameworks, images are represented by tensors. The characteristic dimensions include the *batch size* N_b , defining the number of image examples in the tensor, spatial dimensions N_x, N_y, N_z , temporal dimension N_t , and *feature channels* N_f , which describe the input characteristics of the data. For the multiple building blocks in Section 11.5, we only denote the 1D case if not stated otherwise.

The rawdata $\mathbf{s}$ can easily become high-dimensional according to the dimensionality (2D, ND) of the problem and the underlying imaging application, e.g., motion-resolved imaging, diffusion imaging, and dynamic contrast-enhanced imaging. For example, for a 3D static imaging sequence, the k -space $\mathbf{s}$ would be a tensor of rank 5, i.e., three spatial dimensions, number of MR receiver channels, and two channels to model the complex domain.

Box 11.5 Data formats in ML frameworks

The location of the feature channels vary in different ML frameworks, leading to different data formats. For a 2D tensor, we find following formats:

- Channel first / NCHW: Tensors of size $[N_b, N_f, N_y, N_x]$
- Channel last / NHWC: Tensors of size $[N_b, N_y, N_x, N_f]$

The 3D format can be used to represent both 3D and spatiotemporal 2D+t data:

- Channel first / NCDHW: Tensors of size $[N_b, N_f, N_z, N_y, N_x]$ (3D) or $[N_b, N_f, N_t, N_y, N_x]$ (2D+t)
- Channel last / NDHWC: Tensors of size $[N_b, N_z, N_y, N_x, N_f]$ (3D) or $[N_b, N_t, N_y, N_x, N_f]$ (2D+t)

In Tensorflow / Keras, channel last is the default behavior.

Representation of complex numbers

An interesting aspect in ML MR reconstruction is the choice of how complex-valued MR data is handled. The choice is mainly influenced by the underlying application and architectural design of the specific reconstruction method (see Section 11.4). One way is to view complex-valued MR images as

two-channel real image, which is similar to the processing of RGB images. This mode of operation relies on real-valued calculations, i.e., combines the output of two real-valued operations to a single real-valued output. However, this discards the mathematical relationship between real and imaginary part of the MR signal and thus complex-valued processing is performed only implicitly. On the other hand explicit complex-valued operations on either magnitude/phase or real/imaginary tensors require the consideration of complex calculus.

ML algorithms need to be trained using gradient-based optimization, which requires correct processing of the gradients if complex-valued operations are considered. This requires computing complex derivatives using Wirtinger calculus [76], see Appendix 11.B. For an in-depth explanation on this topic, we refer the interested reader to [76,145,146].

Box 11.6 Implementation of complex-valued input in ML frameworks

Not all layers of NNs do yet support complex data type inputs/outputs. Common practice is therefore to stack the real-valued tensors $\mathbf{x}_i$

- real: $\mathbf{x}_a$, imaginary: $\mathbf{x}_b$
- magnitude: $\mathbf{x}_a$, phase: $\mathbf{x}_b$

into a separate dimension. The choice of the dimension is decided by how the network should process it:

- Real-valued operations: Stack $\mathbf{x}_a$, $\mathbf{x}_b$ tensors into the feature channel dimension N_f , discarding mathematical complex-valued relationship between real and imaginary parts.
- Complex-valued operations (see Appendix 11.B): Stack $\mathbf{x}_a$, $\mathbf{x}_b$ tensors into a separate dimension, requiring consideration of complex-valued layers and complex calculus

Tensorflow ($\geq v1.0$) supports complex-valued processing of the datatypes `tf.complex64` and `tf.complex128`. New complex tensors can be created using `tf.complex(real, imag)`.

Pytorch ($\leq v1.6.0$) does not support complex-valued datatypes. Complex-valued images can be only stored as a two-channel floating-point image. All operations, e.g., complex-valued multiplication, have to be defined manually.

Appendix 11.B Complex calculus

Wirtinger calculus can be leveraged for complex differentiation. Let us assume we have a complex function $f(\mathbf{x}) : \mathbb{C} \rightarrow \mathbb{C}$ where $\mathbf{x} = Re(\mathbf{x}) + iIm(\mathbf{x}) \in \mathbb{C}$ is a scalar. The Wirtinger derivatives are given by a pair of partial derivatives, which are associated with their real and imaginary parts [145]

$$\frac{\partial f}{\partial \mathbf{x}} = \frac{1}{2} \left(\frac{\partial f}{\partial Re(\mathbf{x})} + i \frac{\partial f}{\partial Im(\mathbf{x})} \right) \quad (11.34)$$

$$\frac{\partial f}{\partial \mathbf{x}^H} = \frac{1}{2} \left(\frac{\partial f}{\partial Re(\mathbf{x})} - i \frac{\partial f}{\partial Im(\mathbf{x})} \right). \quad (11.35)$$

The advantage of Wirtinger calculus is that derivatives can be computed directly with respect to $\mathbf{x}$ while $\mathbf{x}^H$ is kept constant, and vice versa. Further important identities of complex derivatives [145] are given

as

$$\frac{\partial f}{\partial \mathbf{x}^H} = \left(\frac{\partial f}{\partial \mathbf{x}} \right)^H \quad (11.36)$$

$$\frac{\partial f}{\partial \mathbf{x}} = \left(\frac{\partial f}{\partial \mathbf{x}^H} \right)^H. \quad (11.37)$$

Following these identities, the chain rule to compute the Wirtinger derivatives of a composite function $f(g(\mathbf{x})) : \mathbb{C} \rightarrow \mathbb{C}$ is given as

$$\frac{\partial f}{\partial \mathbf{x}} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial \mathbf{x}} + \frac{\partial f}{\partial g^H} \left(\frac{\partial g}{\partial \mathbf{x}^H} \right)^H \quad (11.38)$$

$$\frac{\partial f}{\partial \mathbf{x}^H} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial \mathbf{x}^H} + \frac{\partial f}{\partial g^H} \left(\frac{\partial g}{\partial \mathbf{x}} \right)^H. \quad (11.39)$$

For functions that have a real-valued output, i.e., $f(\mathbf{x}) : \mathbb{C} \rightarrow \mathbb{R}$, additional simplifications are given as

$$\frac{\partial f}{\partial \mathbf{x}} = \left(\frac{\partial f}{\partial \mathbf{x}^H} \right)^H \quad (11.40)$$

$$\frac{\partial f}{\partial \mathbf{x}^H} = \left(\frac{\partial f}{\partial \mathbf{x}} \right)^H. \quad (11.41)$$

This results in the simplified chain rule

$$\frac{\partial f}{\partial \mathbf{x}} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial \mathbf{x}} + \left(\frac{\partial f}{\partial g} \right)^H \left(\frac{\partial g}{\partial \mathbf{x}^H} \right)^H \quad (11.42)$$

$$\frac{\partial f}{\partial \mathbf{x}^H} = \left(\frac{\partial f}{\partial g} \right)^H \frac{\partial g}{\partial \mathbf{x}^H} + \frac{\partial f}{\partial g} \left(\frac{\partial g}{\partial \mathbf{x}} \right)^H, \quad (11.43)$$

and depicts the complex back propagation for optimization of a real-valued loss function $\mathcal{L}$ that is computed on the output of an l layer network $f_1 \circ f_2 \circ \dots \circ f_L$, where $\mathbf{x}_l = f_l(\mathbf{x}_{l-1})$, see Fig. 11.13. The update rule for the parameters $\boldsymbol{\theta}_l$ in layer f_l following a gradient descent scheme (see Eq. (11.8)) reads as

$$\begin{aligned} \boldsymbol{\theta}_l^{[it+1]} &= \boldsymbol{\theta}_l^{[it]} - \tau \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_l^H} \\ &= \boldsymbol{\theta}_l^{[it]} - \tau \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L^H} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}^H} \dots \frac{\partial \mathbf{x}_l}{\partial \boldsymbol{\theta}_l^H}. \end{aligned} \quad (11.44)$$

Wirtinger derivatives for common operations and network layers presented in Section 11.5 and are listed in Table 11.3.

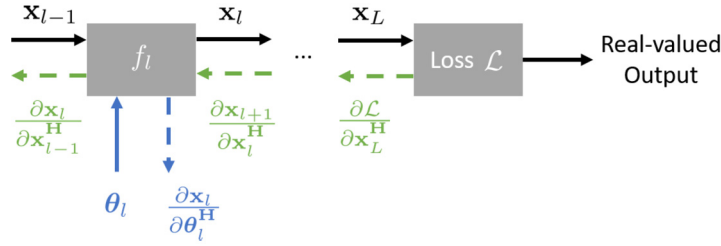


FIGURE 11.13 Complex back-propagation regarding a real-valued loss

To update the network parameters θ , only the adjoint derivatives of the real-valued loss function are required.

Table 11.3 Wirtinger derivatives. Overview of important functions along with their pair of Wirtinger derivatives.

Function	$f(\mathbf{x})$	$\frac{\partial f}{\partial \mathbf{x}}$	$\frac{\partial f}{\partial \mathbf{x}^H}$
Magnitude	$(\mathbf{x}^H \mathbf{x})^{0.5}$	$\frac{\mathbf{x}^H}{2f(\mathbf{x})}$	$\frac{\mathbf{x}}{2f(\mathbf{x})}$
Normalization	$\frac{\mathbf{x}}{(\mathbf{x}^H \mathbf{x})^{0.5}}$	$\frac{1}{2(\mathbf{x}^H \mathbf{x})^{0.5}}$	$-\frac{\mathbf{x}^2}{2(\mathbf{x}^H \mathbf{x})^{1.5}}$
Phase	$-i \log \frac{\mathbf{x}}{ \mathbf{x} }$	$-\frac{i}{2\mathbf{x}}$	$\frac{i}{2\mathbf{x}^H}$
Real Component	$\frac{1}{2}(\mathbf{x} + \mathbf{x}^H)$	$\frac{1}{2}$	$\frac{1}{2}$
Imaginary Component	$\frac{1}{2i}(\mathbf{x} - \mathbf{x}^H)$	$\frac{1}{2i}$	$\frac{i}{2}$
Scalar product	$\mathbf{w}^H \mathbf{x}$	$\mathbf{w}^H$	0
Max Pooling	$\mathbf{x}_n, n = \arg \max_k \mathbf{x}_k $	$\begin{cases} 1 & \text{if } n = \arg \max_k \mathbf{x}_k \\ 0 & \text{else} \end{cases}$	0
Dropout	$\begin{cases} \frac{1}{p} \mathbf{x}_n & \text{if } n \in \Omega \\ 0 & \text{else} \end{cases}$	$\begin{cases} \frac{1}{p} & \text{if } n \in \Omega \\ 0 & \text{else} \end{cases}$	0
Cardioid [76]	$\frac{1}{2}(1 + \cos(\angle \mathbf{x})) \mathbf{x}$	$\frac{1}{2} + \frac{1}{2} \cos(\angle \mathbf{x}) + \frac{i}{4} \sin(\angle \mathbf{x})$	$-\frac{i}{4} \sin(\angle \mathbf{x}) \frac{\mathbf{x}}{\mathbf{x}^H \mathbf{x}}$

Appendix 11.C Trainable parameters of separable convolutions

Depthwise separable convolution is best illustrated in an example. For a normal convolution, let us assume the input of size 32×32 with 3 input channels is convolved with $f = 64$ kernels of size 7×7 (no padding $p = 0$ and stride of $s = 1$) yielding an output of size 26×26 with 64 feature maps. The operation can be split up by first depthwise convolving each input channel independently with a 7×7 convolutional filter ($p = 0, s = 1, f = 1$), i.e., requiring in total 3 2D filters. The output feature maps are stacked along the channel direction giving a tensor of size $26 \times 26 \times 3$ (in channels last notation), which is then convolved pointwise with 64 filters of size $1 \times 1 \times 3$ ($p = 0, s = 1$) producing the final output tensor of size $26 \times 26 \times 64$. The advantage is that fewer multiplications are required in a depthwise separable convolution (3 kernels with $7 \times 7 \times 1$ sliding 32×32 times followed by 64 kernels with $1 \times 1 \times 3$ sliding 32×32 times, yielding $3 \cdot 7 \cdot 7 \cdot 1 \cdot 32 \cdot 32 + 64 \cdot 1 \cdot 1 \cdot 3 \cdot 32 \cdot 32 = 216,064$ multiplications) than in a normal convolution (64 kernels with $7 \times 7 \times 3$ sliding 32×32 times, yielding $64 \cdot 7 \cdot 7 \cdot 3 \cdot 32 \cdot 32 = 9,633,792$ multiplications).

References

- [1] McCarthy J, Minsky ML, Rochester N, Shannon CE. A proposal for the Dartmouth summer research project on artificial intelligence; 2006.
- [2] Rosenblatt F. The perceptron: a probabilistic model for information storage and organization in the brain. *Tech. Rep.* 6. 1958.
- [3] Ackley DH, Hinton GE, Sejnowski J. A learning algorithm for Boltzmann machines. In: *Proceedings of the IEEE conference on decision and control*, vol. 169. IEEE; 1985. p. 147–69.
- [4] Hopfield JJ. Neural networks and physical systems with emergent collective computational abilities. *Proc Natl Acad Sci USA* 1982;79(8):2554–8. <https://doi.org/10.1073/pnas.79.8.2554>.
- [5] Jordan MI. Attractor dynamics and parallelism in a connectionist sequential machine. In: *Proceedings of the eighth annual conference of the cognitive science society*. ISBN 0-8186-2015-3, 1986. p. 531–46.
- [6] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Comput* 1997;9(8):1735–80. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [7] Williams RJ, Zipser D. A learning algorithm for continually running fully recurrent neural networks. *Neural Comput* 1989;1(2):270–80. <https://doi.org/10.1162/neco.1989.1.2.270>.
- [8] Rumelhart DE, Hinton GE, Williams RJ. Learning representations by back-propagating errors. *Nature* 1986;323(6088):533–6. <https://doi.org/10.1038/323533a0>.
- [9] Fukushima K. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol Cybern* 1980;36(4):193–202. <https://doi.org/10.1007/BF00344251>.
- [10] LeCun Y, Haffner P, Bottou L, Bengio Y. Object recognition with gradient-based learning. *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*, vol. 1681. ISBN 3540667229, 1999. p. 319–45.
- [11] Lecun Y, Bengio Y, Hinton G. Deep learning. *Nature* 2015;521(7553):436–44. <https://doi.org/10.1038/nature14539>.
- [12] Krizhevsky A, Sutskever I, Geoffrey EH, Hinton GE. ImageNet classification with deep convolutional neural networks. In: *Advances in neural information processing systems*; 2012. p. 1097–105. Available from: <http://code.google.com/p/cuda-convnet/>.
- [13] Bishop C. *Pattern recognition and machine learning*, vol. 9. Springer; 2006.
- [14] Goodfellow I, Bengio Y, Courville A. *Deep Learning*. MIT Press; 2016.
- [15] Murphy KP. *Machine learning: a probabilistic perspective*. The MIT Press. ISBN 0262018020, 2012.
- [16] Burr S. Active learning literature survey. *Computer sciences technical report*. Tech. Rep. January 2009.
- [17] Kramer MA. Nonlinear principal component analysis using autoassociative neural networks. *AIChe J* 1991;37(2):233–43. <https://doi.org/10.1002/aic.690370209>.
- [18] Kingma DP, Welling M. An introduction to variational autoencoders; 2019.
- [19] Hinton GE, Salakhutdinov RR. Reducing the dimensionality of data with neural networks. *Science* 2006;313(5786):504–7. <https://doi.org/10.1126/science.1127647>.
- [20] Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, et al. Generative adversarial nets. In: *Advances in neural information processing systems*, vol. 3; 2014. p. 2672–80.
- [21] Pan SJ, Yang Q. A survey on transfer learning; 2010.
- [22] Tan C, Sun F, Kong T, Zhang W, Yang C, Liu C. A survey on deep transfer learning. *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*, vol. 11141. Springer Verlag. ISBN 9783030014230, 2018. p. 270–9.
- [23] Raghu M, Zhang C, Kleinberg J, Bengio S. Transfusion: understanding transfer learning for medical imaging. *Tech. Rep.* 2019.
- [24] Brendan McMahan H, Moore E, Ramage D, Hampson S, Agüera y Arcas B. Communication-efficient learning of deep networks from decentralized data. *Tech. Rep.* 2017.
- [25] Yang Q, Liu Y, Chen T, Tong Y. Federated machine learning: concept and applications. *ACM Trans Intell Syst Technol* 2019;10(2):1–19. <https://doi.org/10.1145/3298981>.

- [26] Li T, Sahu AK, Talwalkar A, Smith V. Federated learning: challenges, methods, and future directions. *IEEE Signal Process Mag* 2020;37(3):50–60. <https://doi.org/10.1109/MSP.2020.2975749>.
- [27] Rieke N, Hancox J, Li W, Milletari F, Roth HR, Albarqouni S, et al. The future of digital health with federated learning. *npj Digit Med* 2020;3(1):1–7. <https://doi.org/10.1038/s41746-020-00323-1>.
- [28] Wang Z, Bovik AC, Sheikh HR, Simoncelli EP. Image quality assessment: from error visibility to structural similarity. *IEEE Trans Image Process* 2004;13(4):600–12.
- [29] Zhao H, Gallo O, Frosio I, Kautz J. Loss functions for image restoration with neural networks. *IEEE Trans Comput Imaging* 2016;3(1):47–57.
- [30] Johnson J, Alahi A, Fei-Fei L, Assari SM, Idrees H, Shah M, et al. Perceptual losses for real-time style transfer and super-resolution. In: *Proceedings of the European conference on computer vision*. LNCS, vol. 9906. Springer Verlag. ISBN 9783319464749, 2016. p. 694–711.
- [31] Arjovsky M, Chintala S, Bottou L. Wasserstein generative adversarial networks. In: *Proceedings of the international conference on machine learning*; 2017. p. 214–23.
- [32] Robbins H, Monro S. A stochastic approximation method. *Ann Math Stat* 1951;22(3):400–7. <https://doi.org/10.1214/aoms/1177729586>.
- [33] Richard SS. Two problems with backpropagation and other steepest-descent learning. In: *Procedures for networks* In Proc 8th annual conf cognitive science society, Erlbaum; 1986. p. 823–31.
- [34] Kingma DP, Ba J. Adam: a method for stochastic optimization; 2014. Available from: arXiv:1412.6980.
- [35] Hinton GE, Srivastava NN, Swersky K. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. Tech. Rep. 2012. Available from: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [36] Nesterov Y. A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. *Dokl Akad Nauk USSR* 1983;269:543–7. Available from: <https://ci.nii.ac.jp/naid/20001173129/en/>.
- [37] Duchi J, Hazan E, Singer Y. Adaptive subgradient methods for online learning and stochastic optimization. Tech. Rep. 2010.
- [38] Zeiler MD. ADADELTA: an adaptive learning rate method. Available from: arXiv:1212.5701, 2012.
- [39] Dozat T. Workshop track-ICLR 2016 incorporating Nesterov momentum into ADAM. Tech. Rep. 2016.
- [40] Rudner S. An overview of gradient descent optimization algorithms. Available from: arXiv:1609.04747, 2016.
- [41] LeCun Y, Bottou L, Orr G, Müller KR. Efficient BackProp. *Lecture notes in computer science*, vol. 1524. 1998. p. 5–50.
- [42] Han YS, Yoo J, Ye JC. Deep residual learning for compressed sensing CT reconstruction via persistent homology analysis. Available from: arXiv:1611.06391, 2016.
- [43] Kofler A, Dewey M, Schaeffter T, Wald C, Kolbitsch C. Spatio-temporal deep learning-based undersampling artefact reduction for 2D radial cine MRI with limited training data. *IEEE Trans Med Imaging* 2020;39(3):703–17. <https://doi.org/10.1109/TMI.2019.2930318>.
- [44] Yang G, Yu S, Dong H, Slabaugh G, Dragotti PL, Ye X, et al. DAGAN: deep de-aliasing generative adversarial networks for fast compressed sensing MRI reconstruction. *IEEE Trans Med Imaging* 2017;37(6):1310–21. <https://doi.org/10.1109/tmi.2017.2785879>.
- [45] Hyun CM, Kim HP, Lee SM, Lee SM, Seo JK. Deep learning for undersampled MRI reconstruction. *Phys Med Biol* 2018;63(13):135007. <https://doi.org/10.1088/1361-6560/aac71a>. Available from: https://github.com/hpkim0512/Deep_MRI_Unet.
- [46] Zhu B, Liu JZ, Cauley SF, Rosen BR, Rosen MS. Image reconstruction by domain-transform manifold learning. *Nature* 2018;555(7697):487–92.
- [47] Pruessmann KP, Weiger M, Scheidegger MB, Boesiger P. SENSE: sensitivity encoding for fast MRI. *Magn Reson Med* 1999;42(5):952–62.
- [48] Pruessmann KP, Weiger M, Boernert P, Boesiger P. Advances in sensitivity encoding with arbitrary k-space trajectories. *Magn Reson Med* 2001;46(4):638–51.

- [49] Gregor K, Lecun Y. Learning fast approximations of sparse coding. In: Proceedings of the international conference on machine learning; 2010. p. 399–406.
- [50] Aggarwal HK, Mani MP, Jacob M. Model based image reconstruction using deep learned priors (Modl). In: IEEE international symposium on biomedical imaging; 2018. p. 671–4.
- [51] Schlemper J, Caballero J, Hajnal JV, Price AN, Rueckert D. A deep cascade of convolutional neural networks for dynamic MR image reconstruction. *IEEE Trans Med Imaging* 2018;37(2):491–503. <https://doi.org/10.1109/TMI.2017.2760978>. Available from: <https://github.com/js3611/Deep-MRI-Reconstruction>.
- [52] Duan J, Schlemper J, Qin C, Ouyang C, Bai W, Biffi C, et al. Vs-net: variable splitting network for accelerated parallel MRI reconstruction. *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*, vol. 11767. ISBN 9783030322502, 2019. p. 713–22.
- [53] Adler J, Öktem O. Learned primal-dual reconstruction. *IEEE Trans Med Imaging* 2018;37(6):1322–32. <https://doi.org/10.1109/TMI.2018.2799231>. Available from: https://github.com/adler-j/learned_primal_dual.
- [54] Heide F, Steinberger M, Tsai Yt, Gallo O, Heidrich W, Egiazarian K, et al. FlexISP: a flexible camera image processing framework. *ACM Trans Graph* 2014;33(6):1–13.
- [55] Venkatakrishnan SV, Bouman CA, Wohlberg B. Plug-and-play priors for model based reconstruction. In: IEEE global conference on signal and information processing; 2013. p. 945–8.
- [56] Dabov K, Foi A, Katkovnik V, Egiazarian K. Image denoising with block-matching and 3D filtering. *SPIE electronic imaging*; 2006. p. 606414.
- [57] Buades A, Coll B, Morel JM. Non-local means denoising. *Image Process On Line* 2011;1:208–12.
- [58] Meinhardt T, Moeller M, Hazirbas C, Cremers D. Learning proximal operators: using denoising networks for regularizing inverse imaging problems. In: IEEE international conference on computer vision, vol. 2017-Octob. Institute of Electrical and Electronics Engineers Inc.; 2017. p. 1799–808.
- [59] Kofler A, Haltmeier M, Schaeffter T, Kachelrieß M, Dewey M, Wald C, et al. Neural networks-based regularization for large-scale medical image reconstruction. *Phys Med Biol* 2020;65(13):135003. <https://doi.org/10.1088/1361-6560/ab990e>.
- [60] Ahmad R, Bouman CA, Buzzard GT, Chan S, Liu S, Reehorst ET, et al. Plug-and-play methods for magnetic resonance imaging: using denoisers for image recovery. *IEEE Signal Process Mag* 2020;37(1):105–16. <https://doi.org/10.1109/MSP.2019.2949470>.
- [61] Akçakaya M, Moeller S, Weingärtner S, Uğurbil K. Scan-specific robust artificial-neural-networks for k-space interpolation (RAKI) reconstruction: database-free deep learning for fast imaging. *Magn Reson Med* 2019;81(1):439–53. <https://doi.org/10.1002/mrm.27420>.
- [62] Eo T, Jun Y, Kim T, Jang J, Lee HJ, Hwang D. KIKI-net: cross-domain convolutional neural networks for reconstructing undersampled magnetic resonance images. *Magn Reson Med* 2018;80(5):2188–201. <https://doi.org/10.1002/mrm.27201>.
- [63] Glorot X, Bengio Y. Understanding the difficulty of training deep feedforward neural networks. *Tech. Rep.* 2010.
- [64] He K, Zhang X, Ren S, Sun J. Delving deep into rectifiers: surpassing human-level performance on imagenet classification. In: Proceedings of the IEEE international conference on computer vision, vol. 2015 Inter. ISBN 9781467383912, 2015. p. 1026–34.
- [65] Sandino CM, Cheng JY, Chen F, Mardani M, Pauly JM, Vasanawala SS. Compressed sensing: from research to clinical practice with deep neural networks: shortening scan times for magnetic resonance imaging. *IEEE Signal Process Mag* 2020;37(1):117–27. <https://doi.org/10.1109/MSP.2019.2950433>.
- [66] Küstner T, Fuin N, Hammernik K, Bustin A, Qi H, Hajhosseiny R, et al. CINeNet: deep learning-based 3D cardiac CINE MRI reconstruction with multi-coil complex-valued 4D spatio-temporal convolutions. *Sci Rep* 2020;10(1):1–13. <https://doi.org/10.1038/s41598-020-70551-8>.

- [67] Stergiou A, Poppe R. Spatio-temporal FAST 3D convolutions for human action recognition. In: Proceedings - 18th IEEE international conference on machine learning and applications, ICMLA 2019. Institute of Electrical and Electronics Engineers Inc. ISBN 9781728145495, 2019. p. 183–90.
- [68] Choy C, Gwak J, Savarese S. 4D spatio-temporal convnets: Minkowski convolutional neural networks. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition, vol. 2019-June; 2019. p. 3070–9.
- [69] Wang Y, Song Y, Xie H, Li W, Hu B, Yang G. Reduction of Gibbs artifacts in magnetic resonance imaging based on convolutional neural network. In: Proceedings - 2017 10th international congress on image and signal processing, BioMedical engineering and informatics, CISP-BMEI 2017, vol. 2018-Janua(2); 2018.
- [70] Trabelsi C, Bilaniuk O, Zhang Y, Serdyuk D, Subramanian S, Santos JF, et al. Deep complex networks. In: International conference on learning representations; 2018. Available from: https://github.com/ChihebTrabelsi/deep_complex_networks.
- [71] Ioffe S, Szegedy C. Batch normalization: accelerating deep network training by reducing internal covariate shift. In: Proceedings of the international conference on machine learning; 2015. p. 448–56.
- [72] Ulyanov D, Vedaldi A, Lempitsky V. Instance normalization: the missing ingredient for fast stylization. Available from: arXiv:1607.08022, 2016.
- [73] Ba JL, Kiros JR, Hinton GE. Layer normalization. Available from: arXiv:1607.06450, 2016.
- [74] Wu Y, He K. Group normalization. Tech. Rep. 3. 2020.
- [75] Arjovsky M, Shah A, Bengio Y. Unitary evolution recurrent neural networks. In: Proc. of the international conference on machine learning (ICML); 2016. p. 1120–8.
- [76] Virtue P, Yu SX, Lustig M. Better than real: complex-valued neural nets for MRI fingerprinting. In: IEEE international conference on image processing, vol. 2017-Sept; 2018. p. 3953–7.
- [77] Odena A, Dumoulin V, Olah C. Deconvolution and checkerboard artifacts. Distill 2016. <https://doi.org/10.23915/distill.00003>.
- [78] Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: a simple way to prevent neural networks from overfitting. Tech. Rep. 2014.
- [79] Cho K, Van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: EMNLP 2014 - 2014 conference on empirical methods in natural language processing, proceedings of the conference. Association for Computational Linguistics (ACL). ISBN 9781937284961, 2014. p. 1724–34. Available from: <https://www.aclweb.org/anthology/D14-1179>.
- [80] Wolter M, Yao A. Complex gated recurrent neural networks. In: Advances in neural information processing systems; 2018. p. 10536–46. Available from: <https://github.com/v0lta/Complex-gated-recurrent-neural-networks>.
- [81] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: IEEE conference on computer vision and pattern recognition; 2016. p. 770–8.
- [82] Zagoruyko S, Komodakis N. Wide residual networks. In: British machine vision conference 2016, BMVC 2016, vol. 2016-Sept; 2016. p. 1–87.
- [83] Huang G, Liu Z, Van Der Maaten L, Weinberger KQ. Densely connected convolutional networks. In: Proceedings - 30th IEEE conference on computer vision and pattern recognition, CVPR 2017, vol. 2017-Janua; 2017. p. 2261–9.
- [84] Iandola FN, Han S, Moskewicz MW, Ashraf K, Dally WJ, Keutzer K. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. Available from: arXiv:1602.07360, 2016.
- [85] Hu J, Shen L, Albanie S, Sun G, Wu E. Squeeze-and-excitation networks. IEEE Trans Pattern Anal Mach Intell 2020;42(8):2011–23. <https://doi.org/10.1109/TPAMI.2019.2913372>.
- [86] Zhao L, Li M, Meng D, Li X, Zhang Z, Zhuang Y, et al. Deep convolutional neural networks with merge-and-run mappings. In: IJCAI international joint conference on artificial intelligence, vol. 2018-July. International Joint Conferences on Artificial Intelligence. ISBN 9780999241127, 2018. p. 3170–6.

- [87] Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, et al. Going deeper with convolutions. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition, vol. 07-12-June. IEEE Computer Society. ISBN 9781467369640, 2015. p. 1–9.
- [88] Szegedy C, Vanhoucke V, Ioffe S, Shlens J, Wojna Z. Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition, vol. 2016-Decem. IEEE Computer Society. ISBN 9781467388504, 2016. p. 2818–26.
- [89] Szegedy C, Ioffe S, Vanhoucke V, Alemi AA. Inception-v4, inception-ResNet and the impact of residual connections on learning. In: 31st AAAI conference on artificial intelligence, AAAI 2017. AAAI Press; 2017. p. 4278–84.
- [90] Graves A, Wayne G, Danihelka I. Neural Turing machines. Available from: arXiv:1410.5401, 2014.
- [91] Bahdanau D, Cho KH, Bengio Y. Neural machine translation by jointly learning to align and translate. In: 3rd international conference on learning representations, ICLR 2015 - conference track proceedings. International Conference on Learning Representations, ICLR; 2015. Available from: <https://arxiv.org/abs/1409.0473v7>.
- [92] Luong MT, Pham H, Manning CD. Effective approaches to attention-based neural machine translation. In: Conference proceedings - EMNLP 2015: conference on empirical methods in natural language processing. Association for Computational Linguistics (ACL). ISBN 9781941643327, 2015. p. 1412–21.
- [93] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. Tech. Rep. 2017.
- [94] Cheng J, Dong L, Lapata M. Long short-term memory-networks for machine reading. In: EMNLP 2016 - conference on empirical methods in natural language processing, proceedings, vol. 2(3); 2016. p. 551–61.
- [95] Xu K, Ba JL, Kiros R, Cho K, Courville A, Salakhutdinov R, et al. Show, attend and tell: neural image caption generation with visual attention. In: 32nd international conference on machine learning, ICML 2015, vol. 3. International Machine Learning Society (IMLS). ISBN 9781510810587, 2015. p. 2048–57.
- [96] Qin C, Schlemper J, Caballero J, Price AN, Hajnal JV, Rueckert D. Convolutional recurrent neural networks for dynamic MR image reconstruction. IEEE Trans Med Imaging 2019;38(1):280–90. <https://doi.org/10.1109/TMI.2018.2863670>.
- [97] Hammernik K, Klatzer T, Kobler E, Recht MP, Sodickson DK, Pock T, et al. Learning a variational network for reconstruction of accelerated MRI data. Magn Reson Med 2018;79(6):3055–71. <https://doi.org/10.1002/mrm.26977>. Available from: <https://github.com/VLOGroup/mri-variationalnetwork>.
- [98] Mardani M, Gong E, Cheng JY, Pauly J, Xing L. Recurrent generative adversarial neural networks for compressive imaging. In: IEEE international workshop on computational advances in multi-sensor adaptive processing (CAMSAP); 2017. p. 1–5.
- [99] Lundervold AS, Lundervold A. An overview of deep learning in medical imaging focusing on MRI; 2019.
- [100] Knoll F, Zbontar J, Sriram A, Muckley MJ, Bruno M, Defazio A, et al. fastMRI: a publicly available raw k-space and DICOM dataset of knee images for accelerated MR image reconstruction using machine learning. Radiology Artif Intell 2020;2(1):e190007.
- [101] Lin DJ, Johnson PM, Knoll F, Lui YW. Artificial intelligence for MR image reconstruction: an overview for clinicians; 2020.
- [102] Hammernik K, Knoll F. Machine learning for image reconstruction. In: Rueckert D, Fichtinger G, Zhou SK, editors. Handbook of medical image computing and computer assisted intervention. Elsevier. ISBN 9780128161760, 2020. p. 25–64.
- [103] Ronneberger O, Fischer P, Brox T. U-Net: convolutional networks for biomedical image segmentation. In: International conference on medical image computing and computer assisted intervention; 2015. p. 234–41.
- [104] Han Y, Yoo J, Kim HH, Shin HJ, Sung K, Ye JC. Deep learning with domain adaptation for accelerated projection-reconstruction MR. Magn Reson Med 2018;80(3):1189–205. <https://doi.org/10.1002/mrm.27106>.

- [105] Sandino CM, Lai P, Vasanawala SS, Cheng JY. Accelerating cardiac cine MRI using a deep learning-based ESPIRiT reconstruction. *Magn Reson Med* 2020;mrm.28420. <https://doi.org/10.1002/mrm.28420>. Available from: <http://arxiv.org/abs/1911.05845>. <https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.28420>.
- [106] Chen F, Taviani V, Malkiel I, Cheng JY, Tamir JJ, Shaikh J, et al. Variable-density single-shot fast spin-echo MRI with deep learning reconstruction by using variational networks. *Radiology* 2018;289(2):1–8. <https://doi.org/10.1148/radiol.2018180445>.
- [107] Fuin N, Bustin A, Küstner T, Oksuz I, Clough J, King AP, et al. A multi-scale variational neural network for accelerating motion-compensated whole-heart 3D coronary MR angiography. *Magn Reson Imaging* 2020;70:155–67. <https://doi.org/10.1016/j.mri.2020.04.007>.
- [108] Roth S, Black MJ. Fields of experts. *Int J Comput Vis* 2009;82(2):205–29.
- [109] Mardani M, Gong E, Cheng JY, Vasanawala SS, Zaharchuk G, Xing L, et al. Deep generative adversarial neural networks for compressive sensing (GANCS) MRI. *IEEE Trans Med Imaging* 2019;38(1):167–79. <https://doi.org/10.1109/TMI.2018.2858752>. Available from: <https://github.com/gongenhao/GANCS>.
- [110] Liu F, Samsonov A, Chen L, Kijowski R, Feng L. SANTIS: Sampling-Augmented Neural neTWork with Incoherent Structure for MR image reconstruction. *Magn Reson Med* 2019;82(5):1890–904. <https://doi.org/10.1002/mrm.27827>.
- [111] Aggarwal HK, Mani MP, Jacob M. MoDL: model based deep learning architecture for inverse problems. *IEEE Trans Med Imaging* 2019;38(2):394–405. <https://doi.org/10.1109/TMI.2018.2865356>. Available from: <https://github.com/hkaggarwal/modl>.
- [112] Liu J, Sun Y, Eldeniz C, Gan W, An H, Kamilov US. RARE: image reconstruction using deep priors learned without groundtruth. *IEEE J Sel Top Signal Process* 2020;14(6):1088–99. <https://doi.org/10.1109/jstsp.2020.2998402>.
- [113] Shi J, Liu Q, Wang C, Zhang Q, Ying S, Xu H. Super-resolution reconstruction of MR image with a novel residual learning network algorithm. *Phys Med Biol* 2018;63(8):085011. <https://doi.org/10.1088/1361-6560/aab9e9>.
- [114] Lee D, Yoo J, Tak S, Ye JC. Deep residual learning for accelerated MRI using magnitude and phase networks. *IEEE Trans Biomed Eng* 2018;65(9):1985–95. <https://doi.org/10.1109/TBME.2018.2821699>.
- [115] Wang S, Su Z, Ying L, Peng X, Zhu S, Liang F, et al. Accelerating magnetic resonance imaging via deep learning. In: *IEEE international symposium on biomedical imaging*, vol. 2016-June. IEEE Computer Society. ISBN 9781479923502, 2016. p. 514–7.
- [116] Han Y, Sunwoo L, Ye JC. K-space deep learning for accelerated MRI. *IEEE Trans Med Imaging* 2020;39(2):377–86. <https://doi.org/10.1109/TMI.2019.2927101>.
- [117] Hosseini SAH, Zhang C, Weingärtner S, Moeller S, Stuber M, Ugurbil K, et al. Accelerated coronary MRI with sRAKI: a database-free self-consistent neural network k-space reconstruction for arbitrary undersampling. *PLoS ONE* 2020;15(2):e0229418. <https://doi.org/10.1371/journal.pone.0229418>.
- [118] Cheng J, Mardani M, Alley M, Pauly J, Vasanawala S. DeepSPIRiT: generalized parallel imaging using deep convolutional neural networks. In: *Proceedings of ISMRM 26th annual meeting*; 2018. p. 0570.
- [119] Yang Y, Sun J, Li H, Xu Z. ADMM-Net: a deep learning approach for compressive sensing MRI. In: *Advances in neural information processing systems*; 2017. p. 10–8. Available from: <https://github.com/yangyan92/Deep-ADMM-Net>.
- [120] Vapnik VN. *The nature of statistical learning theory*, No. 4. Springer Science & Business Media; 2013. 409 pp.
- [121] Uv Luxburg, Schölkopf B. *Statistical learning theory: models, concepts, and results*, vol. 10. North-Holland; 2011. p. 651–706.
- [122] IXI dataset. Available from: <https://brain-development.org/ixi-dataset/>.
- [123] ADNI. Available from: <http://adni.loni.usc.edu/>.
- [124] OASIS. Available from: www.oasis-brains.org.
- [125] OpenfMRI. Available from: <http://openfmri.org/>.

- [126] fastMRI. Available from: <https://fastmri.org/dataset/>.
- [127] CHAOS. Available from: <https://chaos.grand-challenge.org/>.
- [128] Deng Jia, Dong Wei, Socher R, Li Li-Jia, Li Kai, Fei-Fei Li. ImageNet: a large-scale hierarchical image database. In: IEEE conference on computer vision and pattern recognition; 2009. p. 248–55.
- [129] Küstner T, Hepp T, Fischer M, Schwartz M, Fritsche A, Häring HU, et al. Fully automated and standardized segmentation of adipose tissue compartments by deep learning in three-dimensional whole-body MRI of epidemiological cohort studies. Available from: arXiv:2008.02251, 2020.
- [130] Dar SUH, Özbey M, Çatlı AB, Çukur T. A transfer-learning approach for accelerated MRI using deep neural networks. *Magn Reson Med* 2020;84(2):663–85. <https://doi.org/10.1002/mrm.28148>.
- [131] Knoll F, Hammernik K, Kobler E, Pock T, Recht MP, Sodickson DK. Assessment of the generalization of learned image reconstruction and the potential for transfer learning. *Magn Reson Med* 2019;81(1):116–28. <https://doi.org/10.1002/mrm.27355>.
- [132] Oksuz I, Ruijsink B, Puyol-Antón E, Clough JR, Cruz G, Bustin A, et al. Automatic CNN-based detection of cardiac MR motion artefacts using k-space data augmentation and curriculum learning. *Med Image Anal* 2019;55:136–47. <https://doi.org/10.1016/j.media.2019.04.009>.
- [133] Chen C, Qin C, Qiu H, Ouyang C, Wang S, Chen L, et al. Realistic adversarial data augmentation for MR image segmentation. In: International conference on medical image computing and computer assisted intervention; 2020. Available from: <http://arxiv.org/abs/2006.13322>.
- [134] Gibson E, Li W, Sudre C, Fidon L, Shakir DI, Wang G, et al. NiftyNet: a deep-learning platform for medical imaging. *Comput Methods Programs Biomed* 2018;158:113–22. <https://doi.org/10.1016/j.cmpb.2018.01.025>.
- [135] Pérez-García F, Sparks R, Ourselin S. TorchIO: a Python library for efficient loading, preprocessing, augmentation and patch-based sampling of medical images in deep learning. Available from: arXiv:2003.04696, 2020.
- [136] Mehrtash A, Pesteie M, Hetherington J, Behringer PA, Kapur T, Wells WM, et al. DeepInfer: open-source deep learning deployment toolkit for image-guided therapy. In: Webster RJ, Fei B, editors. *Medical imaging 2017: image-guided procedures, robotic interventions, and modeling*, vol. 10135. SPIE. ISBN 9781510607156, 2017. p. 101351K.
- [137] Pieper S, Lorensen B, Schroeder W, Kikinis R. The NA-MIC Kit: ITK, VTK, pipelines, grids and 3D slicer as an open platform for the medical image computing community. In: 2006 3rd IEEE international symposium on biomedical imaging: from nano to macro - proceedings, vol. 2006. ISBN 0780395778, 2006. p. 698–701.
- [138] Clarkson MJ, Zombori G, Thompson S, Totz J, Song Y, Espak M, et al. The NifTK software platform for image-guided interventions: platform overview and NiftyLink messaging. *Int J Comput Assisted Radiol Surg* 2015;10(3):301–16. <https://doi.org/10.1007/s11548-014-1124-7>.
- [139] Wolf I, Vetter M, Wegner I, Böttger T, Nolden M, Schöbinger M, et al. The medical imaging interaction toolkit. *Med Image Anal* 2005;9(6):594–604. <https://doi.org/10.1016/j.media.2005.04.005>.
- [140] Nora - the medical imaging platform. Available from: <http://www.nora-imaging.com/>.
- [141] Knoll F, Hammernik K, Zhang C, Moeller S, Pock T, Sodickson DK, et al. Deep-learning methods for parallel magnetic resonance imaging reconstruction: a survey of the current approaches, trends, and issues. *IEEE Signal Process Mag* 2020;37(1):128–40. <https://doi.org/10.1109/MSP.2019.2950640>.
- [142] Wang G, Ye JC, Mueller K, Fessler JA. Image reconstruction is a new frontier of machine learning. *IEEE Trans Med Imaging* 2018;37(6):1289–96.
- [143] Fessler JA. Optimization methods for MR image reconstruction (long version). Available from: arXiv:1903.03510, 2019.
- [144] LeCun Y, Cortes C, Burges C. MNIST handwritten digit database. ATT Labs [Online] 2010;2. Available from: <http://yann.lecun.com/exdb/mnist>.
- [145] Kreutz-Delgado K. The complex gradient operator and the CR-calculus. Available from: arXiv:0906.4835, 2009.
- [146] Wirtinger W. Zur formalen Theorie der Funktionen von mehr komplexen Veränderlichen. *Math Ann* 1927;97(1):357–75. <https://doi.org/10.1007/BF01447872>.